

Programmation Orientée Objets

(Langage C++)

Said Charfi

Maitre de conférences habilité
Département d'Informatique, Faculté des Sciences,
Université Ibn Zohr
Agadir, Maroc

Filières : MIP-I4 et IA4

s.charfi@uiz.ac.ma/charfisaid@gmail.com

- Historique
- Les entrées sorties C++
- Les spécificités du langage C++
- Les classes et Objets
- la sur-définition des opérateurs
- les patrons de fonction et classes
- L'héritage
- Le polymorphisme
- Généralités sur la bibliothèque standard
- La gestion des exceptions

Pour valider le module :

Contrôle des connaissances :

- Un TP et un projet notés : 1/3 de la note finale
- Un examen de fin de semestre : 2/3 de la note finale

Une introduction au langage C++ ainsi qu'au paradigme objet.

L'objectif est de faire découvrir le langage, d'être capable d'écrire et de concevoir un programme C++ simple de bout en bout.

- Claude Delannoy, Apprendre le C++, Eyrolles.
- Claude Delannoy, Programmer en C++, Eyrolles.
- Pour les puristes : Le langage C++ de Bjarne Stroustrup.
- Pour les curieux : LE LANGAGE C. Norme ANSI de Brian W Kernighan , Denis M Ritchie.

Historique

Le C a été inventé au cours de l'année 1972 dans les laboratoires Bell par Dennis Ritchie et Ken Thompson. En 1978 Brian Kernighan qui aida à populariser le C, publia le livre << *The C programming Language* >>, le K&R, qui décrit le C << *traditionnel* >> ou C ANSI.

Dans les années 80 Bjarne Stroustrup développa le C++ afin d'améliorer le C, en lui ajoutant des << *classes* >>. Le premier nom de ce langage fut d'ailleurs << *C with classes* >>. Ce fut en 1998 que le C++ fut normalisé pour la première fois. Une autre norme corrigée fut adoptée en 2003. Une mise à jour importante fut C++ 11 suivie de C++ 14 et C++ 20 ajoutant de nombreuses fonctionnalités au langage. Toutes ces normes permettent une écriture indépendante du compilateur. Le C++ est le même partout, pourvu qu'on respecte ces normes.

Le C++ est une sur-couche de C, avec quelques incompatibilités de syntaxe. Un programme C est la plupart du temps un programme C++. Donc on peut faire du << C+ >> c'est à dire du C++ sans objet.

Impératifs: les instructions se suivent dans un ordre précis et transmis au processeur de la machine dans cet ordre.

Impératif et objet ne se contredisent pas, C++ est un langage multi paradigmes. Il respecte à la fois le paradigme objet et impératif.

On va donc commencer par faire du C++ impératif.

Chapitre 1

Les entrées sorties C++

Les entrées sorties

- Les entrées-sorties en C:

```
#include< stdio.h >          /* #include< cstdio > en C++ */
main(){
int i;
printf(" Donner un entier :");    /* Affichage en C */
scanf( " %d",&i);                /* Lecture en C */
printf( "La valeur tapee est :%d \n", i);    /* Affichage en C */
}
```

- C++ dispose de toutes les routines offertes par la bibliothèque standard du C ANSI (`scanf`, `printf`, ...etc).
- Il comporte de nouvelles possibilités d'entrées-sorties.
- Les entrées-sorties C++ reposent sur les notions de flots et de surdéfinition d'opérateur.
- Dans cette partie, on va se limiter a l'aspect conversationnel :
 - la lecture sur l'entrée standard (généralement le clavier);
 - l'écriture sur la sortie standard (généralement l'écran).

- Exemple 1: La ou en C, on écrivait :

```
printf(" bonjour" );
```

en C++, on utilisera :

```
cout << " bonjour";
```

- << étant un opérateur permettant d'envoyer de l'information sur le flot **cout**, correspondant a l'écran.
- L'opérateur << dispose de deux opérandes :
 - l'opérande de gauche correspond a un flot (plus précisément a un flot de sortie, c'est-a-dire susceptible de recevoir de l'information) ;
 - l'opérande de droite correspond a une expression.

- Exemple 2: Considérons ces instructions :

```
int n = 25 ;
```

```
cout << " valeur : ";
```

```
cout << n;
```

Elles affichent le résultat suivant :

```
valeur : 25
```

- L'opérateur << a été utilisé pour envoyer sur le flot `cout`,
 - une information de type chaine (constante),
 - ensuite une information de type entier
- L'opérateur << est défini pour différents types d'informations (la surdéfinition de <<, voir surcharge des opérateurs).

- L'opérateur `<<` possède une associativité de gauche a droite.

Les deux instructions :

```
cout << " valeur : " ;
```

```
cout << n;
```

peuvent se condenser en une seule :

```
cout << " valeur : " << n;
```

- L'instruction :

```
cout << " valeur : " << n;
```

est équivalente a :

```
(cout << " valeur : ") << n;
```

- Considérons l'exemple :

```
int a;
```

```
cin >> a;
```

- `cin` est un flot d'entrée prédéfini (correspondant au clavier).
- `>>` est un opérateur permettant de lire de l'information sur le flot `cin` correspondant au clavier.
- L'opérateur `<<` dispose de deux opérandes :
 - l'opérande de gauche correspond a un flot d'entrée (susceptible de fournir de l'information) ;
 - l'opérande de droite correspond a une `lvalue`.
- `>>` est défini pour différents types d'informations :

```
int n ;
```

```
char c ;
```

```
.....
```

```
cin << n; /* lit une suite de caractères représentant un entier, la convertit en  
int et range le résultat dans n */
```

```
cin << c; /* lit un caractère et le range dans c */
```

- L'opérateur `>>` possède une associativité de gauche à droite :

`cin >> n >> p;`

sera équivalent à :

`(cin >> n) >> p;`

Comme dans la plupart des cours de programmation, on commence par un HelloWorld. Le Programme le plus simple qui affiche un message à l'écran dans un éditeur de texte quelconque.

```
#include < iostream >
int main()
{
    std :: cout << " HelloWorld" << std :: endl;
}
```

Compilation et exécution du HelloWorld

```
said@said-laptop:/media/Storage/CoursC++  
$ g++ HelloWorld.cpp -o HelloWorld  
said@said-laptop:/media/Storage/CoursC++$  
$ ./HelloWorld.exe  
Hello World
```

Le HelloWorld ligne a ligne

```
#include< iostream >
```

En C++ comme en C, les lignes commençant par sont des << *directives preprocesseurs* >>. Elles s'adressent à un programme appelé préprocesseur, cpp (pour << *c preprocessor* >>) qui prépare le code source en traitant ces directives. Ici, en appelant include on dit au préprocesseur d'inclure le fichier *iostream* de la bibliothèque standard C++ et qui contient les définitions pour afficher quelque chose à l'écran via des << *flots* >>.

Le HelloWorld ligne a ligne

```
int main()
```

Il s'agit de l'entête de la fonction main. En C++, une fonction se divise en deux parties principales : L'entête et le corps de la fonction.

On peut voir ici trois éléments fondamentaux dans l'écriture d'une fonction **main** est le nom de la fonction. En C++, c'est aussi le point d'entrée du programme. Nous verrons plus tard ce que cela signifie. Il faut juste retenir que main est la seule fonction qui doit absolument apparaître dans un programme. C'est une convention.

int est le type de retour de la fonction main, int pour integer c'est à dire que la fonction main, une fois terminée, doit retourner une valeur entière. Pour information, cette valeur peut servir dans l'environnement appelant notre programme une valeur de bonne exécution ou un code erreur.

() ici vide, il s'agit de la liste des arguments fournis lors de l'appel de notre fonction.

Le HelloWorld ligne a ligne

```
{  
std :: cout << " HelloWorld" << std :: endl;  
}
```

Le corps de la fonction main.

Le corps d'une fonction se situe après l'entête de celle-ci et entre deux accolades. Elles définissent en fait le bloc d'instruction de la fonction. Ici, il n'y a qu'une seule instruction, se terminant par un ;

`std::cout` peut être vu comme l'écran, il s'agit du flot de sortie standard.

`<<` est un opérateur opérant sur un flot de sortie à sa gauche et une donnée à lui transmettre, à sa droite.

`<< Hello World >>` est une chaîne de caractère, c'est-à-dire un emplacement mémoire contiguë contenant un caractère par octet de mémoire, et se terminant conventionnellement par le caractère nul. Nous verrons cela plus en détail lorsque nous reparlerons des types de données.

`std::endl` demande au flux de passer à la ligne suivante.

Compilation et exécution du HelloWorld

```
said@said-laptop:/media/Storage/CoursC++  
$ g++ HelloWorld.cpp -o HelloWorld  
said@said-laptop:/media/Storage/CoursC++$  
$ ./HelloWorld.exe  
Hello World
```

Comme il s'agit d'un programme simplissime, la ligne de compilation est elle-même très simple. En la décomposant élément par élément :

g++ est le nom du compilateur c++ de GNU, celui utilisé pour ce cours.

HelloWorld.cpp est le nom du fichier dans lequel on vient d'écrire notre code.

-o HelloWorld est une option transmise au compilateur lui demandant de créer un fichier exécutable portant ce nom là. Il s'agit d'un argument optionnel. Notre programme se nommerait a.out (sous Linux, a.exe sous windows), sans cet argument.

./HelloWorld.exe dans un shell, permet d'exécuter notre programme, qui, comme prévu, affiche Hello World et se termine.

Organisation d'un programme en C++

Le code source d'un programme est un ensemble de fichiers textes qui contiennent les déclarations et les définitions des différents éléments qui seront ensuite transmises au compilateur.

Un fichier se décompose généralement en 2 ou 3 parties.

Les directives préprocesseur (souvenez vous, elles commencent par `#`) se situent généralement en début de fichier.

Viennent ensuite les définitions et déclarations de variables ou de fonctions ou de type de données. Il ne s'agit pas d'instructions à proprement parlé, mais plutôt d'informations qui permettront au compilateur de vérifier la cohérence du code écrit ensuite. Cela peut être assez long, et, souvent le programmeur le déplace dans un fichier header suffixé en `h` ou `hh` qui sera inclus via une directive préprocesseur.

Enfin viennent les définitions des fonctions, le code du programme à proprement parlé, dans un fichier suffixé en `cpp` pour le C++, `c` pour le C.

Organisation d'un programme en C++

```
#include < iostream >
/*Déclaration de la fonction somme*/
double somme(double a, double b);
// point d'entrée de notre programme
int main(){
int a, b;
std :: cout << " Donnez deux entiers" << std ::
endl;
std :: cin >> a >> b;
std :: cout << a << " + " << b << " = " <<
somme(a, b) << std :: endl;
return 0;}
// somme retourne la somme de a et b deux
réels fournis en paramètres.
double somme(double a , double b){
return a+b;}
```

Voici l'exemple d'un programme un peu plus complexe.

On commence par déclarer une fonction somme, sans la définir, c'est-à-dire sans écrire son code. On signifie seulement qu'il existe une telle fonction, prenant deux réels en argument et renvoyant un réel.

On peut à présent l'utiliser dans le code qui suit la déclaration. On pourrait aussi, et on doit même le faire pour plus de propreté, écrire cette partie dans un fichier header.

Organisation d'un programme en C++

```
#ifndef __SOMME_HH__
#define __SOMME_HH__
/*
Déclaration de la fonction
somme
*/
double somme(double a, double
b);
#endif
```

```
#ifndef __SOMME_HH__
#define __SOMME_HH__
```

Un tel fichier header ressemblerait à celui-ci .
On voit apparaître 3 nouvelles directives préprocesseurs ainsi que la déclaration de la fonction somme.
On remarquera aussi que ce fichier NE CONTIENT PAS DE CODE, seulement des déclarations.

Il s'agit simplement d'une protection, évitant lors de programme plus complexe, d'inclure deux fois un fichier header. Le compilateur terminerait alors en erreur car il ne veut pas de multiples définitions (surtout lorsqu'on définira des class). En gros, si la constante `__SOMME_HH__` n'est pas définie, alors inclure le code ci après.

Dans le code en question, on commence par définir une tel constante, et on déclare notre fonction.
Enfin, on ferme la condition `#ifndef` par `#endif`

Tampon et caractères séparateurs

- La première lecture au clavier demande a l'utilisateur de fournir une suite de caractères.
- La suite de caractères doit être validée en tapant la touche entrée (correspond a fin de la ligne).
- La suite est rangée provisoirement dans un emplacement mémoire nommé tampon.
- Le tampon est exploré par >> caractère par caractère chaque fois il y a une lecture.
- Il existe un pointeur qui désigne le prochain caractère a prendre en compte.
- Lors de la prochaine lecture, >> exploite les données disponibles dans le tampon a partir du pointeur. Si le tampon est vide ou il contient moins d'informations a ce que demande par la lecture, >> attendra que l'utilisateur fournisse une nouvelle suite de caractères.
- Dans la suite de caractère, on peut avoir des caractères séparateurs : espace, tabulation.

- Toute lecture commence par avancer le pointeur jusqu'au premier caractère différent d'un séparateur.
- On prend en compte tous les caractères suivants jusqu'à la rencontre d'un séparateur (lorsque aucun caractère invalide n'est présent dans la donnée).
- Supposons **n** et **p** sont de type **int**, tandis que **c** est de type **char**.
- Nous fournissons, pour chaque lecture, des exemples de réponses possibles (^ désigne un espace et @ une fin de ligne) avec les valeurs effectivement lues:

Règles utilisées par >>

Considérons l'instruction *cin* >> *n* >> *p*;

- Lecture 1 :

$12^{25} @ \ n = 12 \ p = 25$

La façon la plus naturelle de fournir les informations voulues.

- Lecture 2 :

$^{12}^{25} @ \ n = 12 \ p = 25$

On a introduit quelques espaces supplémentaires dans les données.

- Lecture 3 :

$12 @$

$@$

$^{25} @ \ n = 12 \ p = 25$

On a fourni trois lignes d'information, dont une "vide". L'opérateur >> a alimenté trois fois le tampon.

- Lecture 4 :

$12^{25} ^{48} ^8 @ \ n = 12 \ p = 25$

L'exploration du tampon s'est arrêtée sur l'espace suivant 25. Les caractères non exploités ici pourront être utilisés par une prochaine lecture.

Règles utilisées par >>

- Pour l'exemple *cin* >> *c* >> *n*;, on aura

$a^{25} @ \quad c = a \quad n = 25$

$a^{25} @ \quad c = a \quad n = 25$

$a^{25} @ \quad c = a \quad n = 25$

- Pour l'exemple *cin* >> *n* >> *c*;, on aura

$12a @ \quad n = 12 \quad c = a$

Manque de synchronisme entre clavier et écran

L'exemple ci-dessous illustre :

- le rôle du tampon,
- comment une lecture peut utiliser une information non exploitée par la lecture précédente (l'utilisateur n'a pas à répondre à la question posée à la troisième ligne).

```
#include< iostream >
using namespace std ;
main(){
  int  n, p;
  cout << " donnez une valeur pour n : ";
  cin >> n;
  cout << " merci pour" << n << " \n";
  cout << " donnez une valeur pour p : ";
  cin >> p;
  cout << " merci pour" << p << " \n";
}
```

exécution :

donnez une valeur pour n : 12
25

merci pour 12

donnez une valeur pour p :
merci pour 25

Blocage de la lecture

```
#include <iostream>
using namespace std ;
main(){
int n = 12;
char c ='a';
cout << " donnez un entier et un caractere :
\n";
cin >> n >> c;
cout << " merci pour" << n << " et" <<
c << " \n";
cout << " donnez un caractere : ";
cin >> c;
cout << " merci pour" << c << endl;
}
```

Sortie :

donnez un entier et un caractère
:
x 25
merci pour 0 et a
donnez un caractère : merci
pour a

- Dans la première lecture de n, l'opérateur >> a rencontré le caractère x qui est invalide.
- >> n'était alors pas capable de fabriquer une valeur entière a copier dans n.
- Le caractère invalide (dans ce cas x) a bloqué la lecture.
- La 2eme tentative de lecture d'un caractère dans c, n'a pas débloqué la situation.

Boucle infinie

```
#include< iostream >
using namespace std ;
main(){
int n ;
do{
cout << " donnez un nombre entier : ";
cin >> n;
cout << " voici son carre : " << n * n << " \ n";
}
while (!n) ;
}
```

donnez un nombre entier :

voici son carre : 0

donnez un nombre entier : voici son carre : 0

donnez un nombre entier : voici son carre : 0

donnez un nombre entier : voici son carre : 0

donnez un nombre entier : voici son carre : 0

Boucle infinie

- Le caractère **a** est invalide pour la fabrication d'un entier.
- La lecture de **n** s'est interrompue et la lecture a été bloquée.
- Les lectures suivantes ont échoué, d'où la boucle infinie.
- Il faudra interrompre l'exécution du programme (exemple `ctr+alt+suppr`).

Le fichier en-tête `iostream`

- En C, l'utilisation des fonctions `printf` et `scanf` nécessitent l'incorporation du fichier `<stdio.h>` `#include<stdio.h>`
- Comparablement les entrées-sorties (E/S) C++ figurent dans un fichier en-tête de nom `<iostream>`.
- Les symboles correspondants sont définis dans l'espace de noms `std` (voir le chapitre sur les espaces de noms).
- Pour qu'on puisse utiliser les E/S C++.

```
#include <iostream>  
using namespace std ; /* on utilisera les symboles définis dans l'espace de noms  
standard s'appelant std */
```

Exercice

Écrire un programme qui demande à l'utilisateur de taper la largeur et la longueur d'un champ et qui en affiche le périmètre et la surface.

```
#include< iostream >
using namespace std;
int main()
{
double largeur,longueur,surface, périmètre;
cout << " Tapez la largeur du champ : ";
cin >> largeur;
cout << " Tapez la longueur du champ : ";
cin >> longueur;
surface = largeur * longueur;
perimetre = 2 * (largeur + longueur);
cout << " La surface vaut : " << surface << endl;
cout << " Le perimetre vaut : " << perimetre << endl;
return 0;
}
```

Les spécificités du langage C++

Introduction

Le langage C++ dispose d'un certain nombre de spécificités qui ne sont pas axées sur la POO:

- Nouvelle forme de commentaire (en fin de ligne).
- La spécification **auto**.
- Emplacement libre des déclarations.
- Notion de référence.
- Arguments par défauts **dans les déclarations des fonctions**.
- Surdéfinitions de fonctions.
- Opérateurs **new** et **delete**.
- Fonctions en "ligne" (**inline**).
- Existence d'un type booléen **bool**.
- Notion d'espace de noms.

Le commentaire de fin de ligne

- En C:

- Un commentaire est mis entre */** et **/*.
- Le commentaire peut s'étaler sur plusieurs lignes.
/ Ceci est un commentaire
suite du commentaire
suite du commentaire */*

- En C++:

- On peut en outre utiliser des commentaires en fin de ligne en introduisant les deux caractères: *//*.
- Tout ce qui est situé entre *//* et la fin de la ligne est un commentaire.
cout << " bonjour n"; //formule de politesse
- On peut toujours écrire:
*cout << " bonjour n"; /*formule de politesse*/*

Le commentaire de fin de ligne

- En C++:

- Melez les deux types de commentaires:

```
/*partie1 // partie2 */ partie3
```

- partie1 et partie2 sont des commentaires puisque elles sont comprises entre `/*` et `*/`

- partie3 considéré comme appartenant aux instructions.

```
partie1 //partie2 /*partie3 */ partie4
```

- Le commentaire introduit par `//` s'étend jusqu'à la fin de la ligne.
- partie2, partie3 et partie4 sont des commentaires.

Déclarations et initialisations

- C++ est plus souple que le C ANSI en matière de déclarations.
- Il n'est plus obligatoire de regrouper les déclarations au début d'une fonction ou d'un bloc.
- Ils peuvent être effectuées n'importe où.
- Ils doivent être déclarées avant leurs utilisations.
- Leur portée reste limitée à la partie du bloc/fonction suivant leur déclaration.

```
void fonction(){
```

```
...
```

```
i = 8; //incorrect
```

```
...
```

```
int i; // déclaration de i
```

```
...
```

```
i = 8; //correct
```

```
...
```

```
{
```

```
...
```

```
float f; //déclaration de f
```

```
...
```

```
f = 4.0; //correct
```

```
...
```

```
} // fin de la portée de f
```

```
...
```

```
f = 4.0; //incorrect
```

```
i = 5; //correct
```

```
} // fin de la portée de i
```

- L'instruction suivante est acceptée en C++

```
for(int i = 0; ...; ...)  
{ ...  
}
```

- La variable `i` a été déclarée au moment où l'on avait besoin.
- Sa portée limitée au bloc régi par l'instruction `for`.

```
for(int i = 0; ...; ...)  
{  
...  
}  
i = 25; // Erreur
```

Déclarations et initialisations:

auto

- Parfois il est difficile de déterminer le type d'une expression qu'on souhaite stocker dans une variable.
- Avec la nouvelle librairie et en utilisant la spécification **auto**, le compilateur déduit le type de donnée à utiliser à partir de son initialisation.

// le type de item est déduit à partir du résultat de l'addition de val1 et val2

auto item = val1 + val2; // item est initialisé à val1 + val2

auto i = 0, *p = &i; // ok: i est un int et p est un pointeur vers un int

auto sz = 0, pi = 3.14; // erreur: types incohérents pour sz et pi

Les références

- Deux façons d'accéder a une donnée représentée en mémoire:

- Utilisation directe de son nom (identificateur)

```
int i;  
i = 2;  
cout << "i = " << i;
```

- Utilisation de son adresse (pointeur)

```
int i;  
int *p = &i;  
*p = 2;  
cout << "i = " << *p;
```

- Le C++ introduit une troisième méthode utilisant la référence
- En C++, une référence est un moyen d'associer un nouveau nom (alias) a un objet existant déjà.
- La référence ne permet pas de créer un nouveau objet.
- La référence (le nouveau nom) devient simplement un synonyme de l'ancien, c'est a dire qu'ils désignent tous deux le même objet.

Définir une référence

```
//définition et initialisation d'une variable  
int unEntier = 4;  
//création d'un synonyme du nom unEntier  
int & aliasDunEntier = unEntier;  
//unEntier vaut maintenant 12  
aliasDunEntier = 12;
```

- Le nom d'une référence désigne l'objet spécifié lors de l'initialisation.
- Pas de changement vis-a-vis de la mémoire (un seul objet référencé par deux noms).
- On ne peut pas définir une référence sans l'initialiser (`int & ref; //erreur`).
- L'objet désigné par une référence ne changera jamais.

Définir une référence

- Pointeur et référence

```
int unEntier = 4;
```

```
int * unPointeur = & unEntier; //& = "adresse de"
```

```
int * & aliasDunPointeur = unPointeur;
```

- Ne pas confondre pointeur et référence!

- L'accès a une variable:

```
int uneVariable = 4;
```

```
int * unPointeur = & uneVariable;
```

```
int &uneReference = uneVariable;
```

```
//On peut maintenant accéder a uneVariable en utilisant:
```

```
uneVariable = 5; //son nom
```

```
*unPointeur = 6; //un pointeur contenant son adresse
```

```
uneReference = 7; //une référence
```

Pourquoi utiliser une référence ?

```
int uneVariable = 4;  
int * unPointeur = & uneVariable;  
int &uneReference = uneVariable;  
//On peut maintenant acceder a uneVariable en utilisant:  
uneVariable = 5; //son nom  
*unPointeur = 6; //un pointeur contenant son adresse  
uneReference = 7; //une reference
```

- C'est la question qui peut se poser en regardant l'exemple précédent, où il serait plus clair d'utiliser directement les variables.
- Les références sont principalement utilisées pour passer des paramètres aux fonctions (voir passage de paramètres par référence).

Transmission des arguments en C: Par valeur

- En C, les arguments d'une fonction sont transmis par valeur.

```
#include< iostream >
using namespace std;
void echange(int, int);
main()
{ int n=10, p=20;
  cout << "avant appel : ";
  cout << n << " " << p << endl;
  echange(n, p);
  cout << "apres appel : ";
  cout << n << " " << p << endl;
}
```

```
void echange(int a, int b)
{
  int c;
  cout << "debut echange : ";
  cout << a << " " << b << endl;
  c = a; a = b; b = c;
  cout << "fin echange : ";
  cout << a << " " << b << endl;
}
```

Sortie

```
avant appel : 10 20
debut echange : 10 20
fin echange : 20 10
apres appel : 10 20
```

- La fonction `echange(int,int)` n'a pas réussi à permuter les valeurs des deux entiers `n` et `p`.

Transmission des arguments en C: Par adresse

- Afin de modifier la valeur d'une variable, il faut transmettre son adresse.

```
#include< iostream >
using namespace std;
void echange(int*, int*);
main()
{ int n=10, p=20;
  cout << "avant appel : ";
  cout << n << " " << p << endl;
  echange(&n, &p);
  cout << "apres appel : ";
  cout << n << " " << p << endl;
}
```

```
void echange(int *a, int *b)
{
  int c;
  cout << "debut echange : ";
  cout << *a << " " << *b << endl;
  c = *a; *a = *b; *b = c;
  cout << "fin echange : ";
  cout << *a << " " << *b << endl;
}
```

Sortie

```
avant appel : 10 20
debut echange : 10 20
fin echange : 20 10
apres appel : 20 10
```

- La fonction `echange(int*,int*)` a réussi a permuter les valeurs des deux entiers n et p.

Transmission par référence en C++

```
#include< iostream >
using namespace std;
void echange(int&, int&);
main()
{ int n=10, p=20;
  cout << "avant appel : ";
  cout << n << " " << p << endl;
  echange(n, p);
  cout << "apres appel : ";
  cout << n << " " << p << endl;
}
```

```
void echange(int &a, int &b)
{
  int c;
  cout << "debut echange : ";
  cout << a << " " << b << endl;
  c = a; a = b; b = c;
  cout << "fin echange : ";
  cout << a << " " << b << endl;
}
```

Sortie

```
avant appel : 10 20
debut echange : 10 20
fin echange : 20 10
apres appel : 20 10
```

- La fonction `echange(int&,int&)` a réussi a permuter les valeurs des deux entiers n et p.

Transmission par référence

- Absence de conversion

```
void fonction(int &n);  
float x;
```

...

```
f(x); // illégal
```

- Argument effectif constant

```
void fct(int &);  
fct(3); //incorrect  
const int c = 15;  
fct(c); //incorrect
```

- fct ne peut pas modifier une constante.

Transmission par référence

- Argument muet constant

```
void fct1(const int &);  
const int c=15;  
fct1(3); //correct  
fct1(c); //correct
```

- La déclaration `const int &` correspond a une référence a une constante.
- Un appel tel `fct1(expression)` sera accepté quelque soit le type de `expression`.
- Création d'une variable temporaire (de type `int`) qui recevra le résultat de la conversion de `expression` en `int`.
- La fonction reçoit la référence a la variable temporaire contenant le résultat de la conversion de `expression` en `int`.
- Par exemple:

```
void fct1(const int &);  
float x;  
...  
fct1(x); //correct
```

Transmission par référence d'une valeur de retour

```
int & f()  
{  
...  
return n; // on suppose n de type int  
}
```

- Un appel de **f** provoquera la transmission de la référence de **n**, non plus sa valeur.
- Exemple d'appel:

```
int p;  
...  
p = f();
```
- y a-t-il un intérêt de la transmission par référence par rapport a celle par valeur?
 - Non!
- Il est nécessaire que **n** ne soit pas locale (on peut pas récupérer une référence (adresse) a quelque chose qui n'existe pas).

Transmission par référence d'une valeur de retour:

On obtient une **lvalue**

- On peut utiliser l'appel de la fonction f comme **lvalue**.

```
int & f ();  
int n;  
float x;  
...  
f() = 2*n+5; //a la référence fournie par f,  
//on range la valeur de l'expression 2*n+5  
//de type int  
f() = x; //a la référence fournie par f,  
//on range la valeur de x après  
//conversion en int
```

- On va bien voir l'intérêt de la transmission par référence lors de la surdéfinition des opérateurs (L'opérateur [] par exemple).

Les arguments par défaut

- En C ANSI, il est indispensable que l'appel d'une fonction contienne autant d'arguments que la fonction en attend effectivement.
- C++ permet s'affranchir de cette règle grâce à une attribution de valeurs par défaut à des arguments.

Les arguments par défaut: Exemple 1

```
void fct(int, int=12); // proto avec une valeur par défaut
main(){
  int n=10, p=20;
  fct(n,p); //appel normal
  fct(n); //appel avec un seul argument
  // fct() serait rejeté
}
void fct(int a, int b){ //en-tête habituelle
  cout << " premier argument : " << a << " \n";
  cout << " second argument : " << b << " \n";
}
```

Sortie

```
premier argument : 10
second argument : 20
premier argument : 10
second argument : 12
```


Les arguments par défaut: Exemple 2

```
void fct(int=0, int=12); // proto avec une valeur par défaut
main(){
  int n=10, p=20;
  fct(n,p); //appel normal
  fct(n); //appel avec un seul argument
  fct(); // appel sans argument
}
void fct(int a, int b) { //en-tête habituelle
  cout << "premier argument : " << a << " \n";
  cout << "second argument : " << b << " \n";
}
```

Sortie

```
premier argument : 10
second argument : 20
premier argument : 10
second argument : 12
premier argument : 0
second argument : 12
```

Les propriétés des arguments par défaut

- Les arguments présentant des valeurs par défaut doivent obligatoirement être les derniers de la liste.
 - La déclaration telle que :
`float fexple(int=5, long, int=3);`
est interdite.
- Les valeurs par défaut sont à fixer dans la déclaration de la fonction et non dans sa définition.
- Dans l'attribution d'une valeur par défaut à un argument de type pointeur, il faut que `*` et `=` soient séparés par au moins un espace.
Sinon, ils seraient interprétés comme l'opérateur `*=`.
- Les valeurs par défaut ne sont pas nécessairement des expressions constantes.
- Elles ne peuvent pas faire intervenir de variables locales.

Surdéfinition de fonctions

- On parle de surdéfinition lorsqu'un même symbole possède plusieurs significations différentes.
- Le choix d'une signification se faisant en fonction du contexte.
- Il y a aussi la surdéfinition des opérateurs (voir plus loin).
 - Il repose sur la surdéfinition de fonctions.
- La surdéfinition de fonctions:
 - Employer plusieurs fonctions de même nom dans le même programme.
 - La différence entre ces fonctions réside dans le nombre et/ou type d'arguments.

Surdéfinition de fonctions : Exemple 1

```
#include< iostream >
using namespace std ;
void saisie (int) ; // les prototypes
void saisie (double) ;
main() // le programme de test
{ int n=5 ;
  double x=2.5 ;
  saisie(n) ;
  saisie(x) ;
}
void saisie(int a) // la première fonction
{
  cout << "saisie numero I a =" << a <<
  " \ n";
}
void saisie(double a) // la deuxième fonction
{
  cout << "saisie numero II a =" << a <<
  " \ n";}
```

Sortie

saisie numero I a = 5
saisie numero II a = 2.5

- Le compilateur a bien mis en place l'appel de la bonne fonction saisie en se basant sur le type de l'argument.
- Exemple simple, le type d'argument effectif est le même que celui muet.

Surdéfinition de fonctions

Choix d'une fonction surdéfinie

- Considérons:
`void saisie (int) ;`
`void saisie (double) ;`
`char c ; float y ;`
- Que se passe t il si on appelle avec un argument de type différent de int et double?
 - `saisie(c) ;` // appelle saisie I, après conversion de c en int
 - `saisie(y) ;` // appelle saisie II, après conversion de y en double
 - `saisie('d') ;` // appelle saisie I, après conversion de 'd' en int

Surdéfinition de fonctions

Choix d'une fonction surdéfinie

- Exemple-2

```
void essai (int, double) ; // essai I  
void essai (double, int) ; // essai II  
int n, p ; double z ; char c ;
```

- Appel des fonctions surdéfinies:

- `essai(n,z) ;`
// appelle essai I
- `essai(c,z) ;`
// appelle essai I, après conversion de c en int
- `essai(n,p) ;`
// erreur de compilation,
- Ambiguïté: convertir `p` en double sans modifier `n` et appeler essai I ou, au contraire, convertir `n` en double sans modifier `p` et appeler essai II.

Surdéfinition de fonctions

Choix d'une fonction surdéfinie

- Exemple-3:

```
void test (int n=0, double x=0) ; // test I  
void test (double y=0, int p=0) ; // test II  
int n ; double z ;
```

- Appel des fonctions surdefinies:

- `test(n,z) ;` // appelle test I
- `test(z,n) ;` // appelle test II
- `test(n) ;` // appelle test I
- `test(z) ;` // appelle test II
- `test() ;` // erreur de compilation
- Possibilité d'appeler les deux fonctions (ambiguïté).

Surdéfinition de fonctions

Choix d'une fonction surdéfinie

- Exemple-4:

```
void truc (int) ; // truc I  
void truc (const int) ; // truc II
```

- On obtient une erreur de compilation.
- C++ ne distingue pas `int` et `const int`.
- Les deux fonctions recevant une copie de l'information à traiter.
- Aucun risque de modifier la valeur originale.
- L'erreur est générée à cause de la présence des deux fonctions (indépendamment d'un appel quelconque).

Surdéfinition de fonctions

Choix d'une fonction surdéfinie (Exemple 4)

```
1 // ref1.cpp
2 #include< iostream >
3 using namespace std;
4 void truc (int) ; // truc I
5 void truc (const int) ; // truc II
6 main()
7 { // pas d'appel aux fonctions }
8 void truc (int) {
9     cout << "I \ n";}
10 void truc (const int) {
11     cout << "II \ n";
12 }
```

Output

```
said@said-laptop:/media/Storage/CoursC++$ g++ -c ref1.cpp
ref1.cpp: In function void truc(int):
ref1.cpp:10:6: erreur: redefinition of void truc(int)
ref1.cpp:8:6: erreur: void truc(int) previously defined here
```

Surdéfinition de fonctions

Choix d'une fonction surdéfinie (Exemple 5)

- Considérons ce bloc d'instructions:

```
void chose (int &) ; // chose I
void chose (const int &) ; // chose II
int n = 3 ;
const int p = 5 ;
.....
chose (n) ; // appelle chose I
chose (p) ; // appelle chose II
```

- La distinction entre `int &` et `const int &` est justifiée.
 - La fonction `chose I` modifie la valeur de la variable transmise en argument par référence.
 - La fonction `chose II` n'en fait rien.

Surdéfinition de fonctions

Choix d'une fonction surdéfinie (Exemple 6)

- Considérons le bloc

```
void chose (int &) ; // chose I  
void chose (const int &) // chose II  
int n;  
float x ;
```

- L'appel: `chose (n) ;`
 - appelle la fonction chose I
- L'appel : `chose (2) ;`
 - appelle la fonction chose II, après copie éventuelle de 2 dans un entier temporaire dont la référence sera transmise a chose
- L'appel: `chose (x) ;`
 - appelle chose II, après conversion de la valeur de x en un entier temporaire dont la référence sera transmise a chose

Surdéfinition de fonctions

Choix d'une fonction surdéfinie (Exemple 7)

- Considerons le bloc d'instructions

```
void chose(int*); //chose I  
void chose(const int *); //chose II  
int n = 3;  
const int p =5;
```

- L'appel : chose(&n);
 - appelle chose I
- L'appel : chose(&p);
 - appelle chose II
- Il y a distinction entre `int *` et `const int *`

Surdéfinition de fonctions

Remarques

- Le mode de transmission (référence ou valeur) n'intervient pas dans le choix d'une fonction surdefinie.
- Comme exemple, les déclarations
`void chose (int &) ;`
`void chose (int) ;`
conduiraient a une erreur de compilation due a leur ambiguïté (dans l'appel).
- La surdéfinition de fonctions s'effectue sur la base de:
 - nombre d'arguments
 - leur type
- Le type de retour n'intervient pas en surdéfinition de fonctions.
- `int &` se distingue de `const int &`.
- `int *` se distingue de `const int *`.

Règles de recherche d'une fonction surdéfinie

Cas des fonctions a un argument

- Le compilateur recherche la meilleure correspondance possible.
- Il faut que le compilateur dispose d'un critère d'évaluation.
- Il est prévu différents niveaux de correspondance:
 - 1 Correspondance exacte
 - Il y a correspondance exacte entre les arguments effectifs et ceux muets.
 - 2 Correspondance avec promotions numériques
 - char et short sont convertis en int
 - float est converti en double
 - 3 Conversions dites standard
 - Les conversions légales en C++ (imposées par une affectation).
 - Toute conversion d'un type numérique en un autre type numérique est acceptée.
 - Il peut s'agir de conversions dégradantes (Exemple: la conversion de float en double).

Cas des fonctions a un argument

- La recherche s'arrête au premier niveau ayant permis de trouver une correspondance, qui doit alors être unique.
- Si plusieurs fonctions conviennent au même niveau de correspondance, il y a erreur de compilation due à l'ambiguïté rencontrée.
- Si aucune fonction ne convient à aucun niveau, il y a aussi erreur de compilation.

Cas des fonctions a plusieurs arguments

- L'idée consiste a trouver une fonction meilleure que toutes les autres.
- Comment?
 - 1 Le compilateur sélectionne, pour chaque argument, la ou les fonctions qui réalisent la meilleure correspondance (au sens de la hiérarchie définie ci-dessus) .
 - 2 Parmi les fonctions sélectionnées, il choisit celle qui réalise, pour chaque argument, une correspondance au moins égale a celle de toutes les autres fonctions.
 - 3 Si plusieurs fonctions conviennent, on aura une erreur de compilation due a l'ambigüité rencontrée.
 - 4 Si aucune fonction ne convient, il y aura erreur de compilation.

Les opérateurs **new** et **delete**

- En langage C, la gestion dynamique fait appel a des fonctions de la bibliothèque standard:
 - Allocation: **malloc, calloc, realloc**
 - Deallocation: **free**
- Ces fonctions restent utilisables en C++.
- Le langage C++ a introduit deux nouveaux opérateurs pour la gestion dynamique:
 - Allocation: **new**.
 - Deallocation: **delete**.

Exemple 1:

- Avec la déclaration

```
int *ad;
```

l'instruction :

```
ad = new int;
```

permet d'allouer l'espace mémoire nécessaire pour un élément de type **int** et d'affecter a **ad** l'adresse correspondante.

- Son équivalent en C:

```
ad = (int *) malloc(sizeof(int));
```

- On peut déclarer la variable ad ou on veut en écrivant:

```
int * ad = new int;
```

Exemple 2:

- Avec la déclaration :

```
char *adc ;
```

l'instruction :

```
adc = new char[100] ;
```

alloue l'emplacement nécessaire pour un tableau de 100 caractères et place l'adresse (de début) dans **adc**.

- En C:

```
adc = (char *) malloc(100*sizeof(char);
```

ou

```
adc = (char *) malloc(100); // Parce que sizeof(char)  
// donne 1
```

- L'opérateur unaire **new** s'utilise ainsi :

new type

- **type** représente un type quelconque.
- Il fournit comme résultat un pointeur (de type **type ***) sur l'emplacement correspondant, si l'allocation a réussi.

- L'opérateur **new** accepte également une syntaxe de la forme :

new type [n]

- **n** désigne une expression entière quelconque (non négative).
- Cette instruction alloue alors l'emplacement nécessaire pour **n** éléments du type indique.
- Si l'opération a réussi, elle fournit en résultat un pointeur (toujours de type **type ***) sur le premier élément de ce tableau.

- Echec d'allocation avec **new**.
 - **new** déclenche une exception de type **bad_alloc** (voir gestion des exceptions).
 - Vous verrez que si rien n'est prévu par le programmeur pour traiter une exception, le programme s'interrompt.
 - Il est cependant possible de demander a **new** de se comporter différemment en cas d'échec.
- **new** peut être utilisé pour allouer un emplacement pour un tableau a plusieurs dimensions, par exemple (Deux dimensions):
new type [n] [10]
 - **new** fournit un pointeur sur des tableaux de 10 entiers.
 - la première dimension peut être une expression entière quelconque;
 - les autres dimensions doivent obligatoirement être des expressions constantes.
 - cette possibilité est rarement utilisée en pratique.

L'opérateur `new(throw)`

- En cas d'échec, `new` déclenche une exception `bad_alloc`.
- Dans les versions d'avant la norme, `new` fournissait un pointeur nul en cas d'échec.
- Avec la norme, on peut retrouver ce comportement en utilisant, au lieu de `new`, l'opérateur `new (nothrow)` ou `new(std::nothrow)`.

```
main(){  
    long taille ;  
    int * adr ;  
    int nbloc ;  
    cout << " Taille souhaitee?";  
    cin >> taille;  
    for (nbloc=1 ; ; nbloc++)  
    { adr = new (nothrow) int [taille] ;  
    if (adr==0) { cout <<  
        " ****manque de memoire****\n";  
        exit (-1) ;  
    }  
    cout << " Allocation bloc numero :  
    " << nbloc << " \n";}
```

Output

```
Taille souhaitee ? 4000000  
Allocation bloc numero : 1  
Allocation bloc numero : 2  
Allocation bloc numero : 3  
**** manque de mémoire ****
```

L'opérateur `new(throw)`

- Si on utilise `new` au lieu de `new (nothrow)` ou `new(std::nothrow)`.
- `new` déclenche une exception `bad_alloc`.

```
main(){
long taille ;
int * adr ;
int nbloc ;
cout << " Taille souhaitee?";
cin >> taille;
for (nbloc=1 ; ; nbloc++)
{ adr = new int [taille] ;
if (adr==0) { cout <<
" ****manque de memoire****\n";
exit (-1) ;
}
cout << " Allocation bloc numero :
" << nbloc << " \n";
}
}
```

Output

```
Taille souhaitee ? 4000000
Allocation bloc numero : 1
Allocation bloc numero : 2
Allocation bloc numero : 3
terminate called after throwing
an instance of 'std::bad_alloc'
what(): std::bad_alloc
Abandon (core dumped)
```

Gestion des débordements de mémoire avec `set_new_handler`

- On peut gérer le débordement mémoire autrement.
 - Possible de définir une fonction de votre choix qui soit appelée en cas de manque mémoire.

```
void deborde () // fonction appelée en cas de manque mémoire
{
    cout << " memoire insuffisante \n";
    cout << " Abandon de l'execution \n";
    exit (0) ;
}
```

- Appeler la fonction `set_new_handler` qui reçoit comme entrée la fonction prévue pour la gestion mémoire (définie par le programmeur).

```
#include < new > // pour set_new_handler
```

...

```
set_new_handler (deborde) ; //appel de cette fonction dans le programme
// ou il y a allocation (new)
```


Gestion des débordements de mémoire avec `set_new_handler`

```
#include < cstdlib > // pour exit
#include < new > // pour set_new_handler
#include < iostream >
using namespace std ;
void deborde () ; // proto fonction appelée en cas manque mémoire
main(){ // programme principal
set_new_handler (deborde) ;
long taille ;
int * adr ;
int nbloc ;
cout << " Taille de bloc souhaitée (en entiers) ?";
cin >> taille;
for (nbloc=1 ; ; nbloc++){ adr = new int [taille] ;
cout << " Allocation bloc numero : " << nbloc << " \ n";
} }
```

Gestion des débordements de mémoire avec `set_new_handler`

```
// definition de la fonction
void deborde () // fonction appelée en cas de manque mémoire
{
    cout << "memoire insuffisante \n";
    cout << "Abandon de l'execution \n";
    exit (0);
}
```

Output

```
Taille de bloc souhaitee (en entiers) ? 200000000
Allocation bloc numero : 1
Allocation bloc numero : 2
Allocation bloc numero : 3
mémoire insuffisante
Abandon de lexécution
```

L'opérateur `delete`

- `delete` est utilisé pour libérer un emplacement alloué préalablement par `new`.
- pour l'emplacement alloué par
`int * ad;`
`ad = new int;`
on utilise `delete ad;` pour sa libération.
- de même pour
`char * adc;`
`adc = new char[100];`
on utilise `delete adc;`
- La syntaxe usuelle de l'opérateur `delete` est la suivante :
`delete adresse;`
- `adresse` étant une expression devant avoir comme valeur un pointeur sur un emplacement alloué par `new`.
- Remarque: Il existe une autre syntaxe de `delete` qui n'intervient que dans le cas de tableaux d'objets :
`delete [] adresse;`

Exemple (new, delete)

```
#include< iostream >
main(){
using namespace std;
int *adi, *adibis;
int nb ;
cout << " combien de valeurs : ";
cin >> nb;
// allocation d'un emplacement pour nb entiers dans lesquels
// on place les carres des nombres de 1 a nb
adi = new int [nb] ;
cout << " allocation de " << nb << " int en : " << adi << " \n";
//remplissage du tableau
for (int i=0 ; i<nb ; i++) *(adi+i) = (i+1)*(i+1) ;
// affichage des éléments du tableau
for (adibis = adi ; adibis<adi+nb ; adibis++) cout << *adibis << " ";
cout << " \n";
// libration des nb int
delete adi ;
cout << " liberation des " << nb << " int en : " << adi << " \n";}
```

Exemple (new, delete)

- Compilation:

```
said@laptop:/CoursC++$ g++ -o sortie Exp_new_delete.cpp
```

- exécution:

```
said@laptop:/CoursC++$ ./sortie
```

```
combien de valeurs : 4
```

```
allocation de 4 int en : 0x8fe6008
```

```
1 4 9 16
```

```
liberation des 4 int en : 0x8fe6008
```

```
said@laptop:/CoursC++$
```

La spécification **inline**

Les macros et les fonctions

- En C, il existe deux notions assez voisines a savoir les macros et les fonctions.
- Les macros:

```
#define carre(x) x*x // introduction de la macro
main(){
int a = 2 , b;
b = carre(a); // appel de la macro
}
```

- Les fonctions:

```
int carre(int x); // déclaration de la fonction
main(){
int a = 2 , b;
b = carre(a); // appel de la fonction
}
int carre(int x) // définition de la fonction
{
return x*x;
}
```

La spécification **inline**

Les macros et les fonctions

Les macros

- Le code correspondant est introduit a chaque appel (au niveau du préprocesseur et non plus au niveau du compilateur).
 - L'instruction
b = carre(a);
va être remplacée par **b = a*a;**
dans l'étape de précompilation.

Programme initial:

```
#define carre(x) x*x
main(){
int a = 2 , b;
b = carre(a);
}
```

Programme après précompilation:

```
#define carre(x) x*x
main(){
int a = 2 , b;
b = a*a;
}
```

- On obtient un gain de temps d'exécution, en contrepartie d'une perte d'espace mémoire.

La spécification **inline**

Les macros et les fonctions

Les fonctions

- Le code exécutable correspondant a une fonction est génère une seule fois par le compilateur.
- L'appel de cette fonction s'effectue en suivant les étapes suivantes:
 - ① sauvegarde de l'état courant (valeurs de certains registres de la machine par exemple) ;
 - ② allocation d'espace sur la pile et copie des valeurs des arguments ;
 - ③ branchement a la fonction (dont l'adresse définitive sera en fait fournie par l'éditeur de liens) ;
 - ④ recopie de la valeur de retour ;
 - ⑤ restauration de l'état courant et retour dans le programme appelant.
- Les fonctions sont moins rapide que les macros : l'appel d'une fonction nécessite l'exécution des instructions ci-dessus (1 a 5).
- Les fonctions permettent de gagner en espace contrairement aux macros.

La spécification **inline**

Les macros

Problème de bord.

- Les macros peuvent entrainer des effets de bord indésirables (imprévus), contrairement aux fonctions.
- Il est souvent nécessaire de prendre quelques précautions, notamment lorsque le texte de substitution fait intervenir des opérateurs.
- Par exemple, avec ces instructions :

```
#define DOUBLE(x) x+x
```

```
.....
```

```
DOUBLE(a)/b
```

```
DOUBLE(x+2*y)
```

```
DOUBLE(x++)
```

Le texte génère par le préprocesseur sera le suivant :

```
a + a/b // au lieu de (a + a)/b
```

```
x+2*y + x+2*y // correct
```

```
x++ + x++ // incrémentation deux fois de x
```

La spécification **inline**

Les macros

Exemple de Problème de bord:

```
#include< iostream >
using namespace std;
#define double(x) x+x
main(){
float a = 2, b=3 ;
float res ;
res = double(a)/b;
cout << "res  = " << res << endl;
res = double(a + 2*b);
cout << "res  = " << res << endl;
res = double(a++);
cout << "res  = " << res << endl;
cout << "a   = " << a << endl;
}
```

Sortie

```
res = 2.66667
res = 16
res = 5
a = 4
```

- $\text{double}(a)/b$ a été remplacée par $a + a/b = 2 + 2/3 = 2.66..$ au lieu de $(a + a)/b = (2 + 2)/3 = 1.33..$
- Pas de Problème au niveau de $\text{double}(a + 2*b)$
- $\text{double}(a++)$ a été remplacée par $a++ + a++$. Le résultat obtenu est : $\text{res} = 5$ et $a = 4$ au lieu de $\text{res} = 4$ et $a = 3$

La spécification **inline**

Les macros

Problème de bord:

- Le Problème lie aux priorités relatives des opérateurs peut être résolu en introduisant des parenthèses dans la définition de la macro.

```
#define DOUBLE(x) ((x)+(x))
```

```
...
```

```
DOUBLE(a)/b
```

```
DOUBLE(x+2*y)
```

```
DOUBLE(x++)
```

- Le texte généré par le préprocesseur sera

```
((a)+(a))/b
```

```
((x+2*y)+(x+2*y))
```

```
((x++)+(x++))
```

- L'effet de bord introduit par la troisième instruction (incrément) reste toujours pose.

- En C, lorsque on a besoin d'une fonction courte et que le temps d'exécution est primordial
 - on utilise les macros.
 - les macros présentent des inconvénients!
- En C++, il existe une solution plus satisfaisante : utilisation des fonction en ligne (**inline**).
- Une fonction en ligne se définit et s'utilise comme une fonction ordinaire.
- La seule différence avec les fonctions ordinaires, on fait précéder l'en-tête de la fonction en ligne de la spécification **inline**.

La spécification **inline**

Exemple

```
#include< cmath > // ancien < math.h > pour sqrt
#include< iostream >
using namespace std ;
/* définition d'une fonction en ligne */
inline double norme (double vec[3])
{ int i ; double s = 0 ;
  for (i=0 ; i<3 ; i++)
    s+= vec[i] * vec[i] ;
  return sqrt(s) ;}
/* exemple d'utilisation */
main(){
  double v1[3], v2[3] ;
  int i ;
  for (i=0 ; i<3 ; i++){
    v1[i] = i ; v2[i] = 2*i-1 ;}
  cout << "norme de v1 : " << norme(v1) <<
    "\n" ;
  cout << "norme de v2 : " << norme(v2) <<
    "\n" ;}
```

exécution

norme de v1 : 2.23607

norme de v2 : 3.31662

- La fonction **norme** a pour but de calculer la norme d'un vecteur a trois composantes.
- La présence du mot **inline** demande au compilateur de traiter la fonction norme différemment d'une fonction ordinaire.
- A chaque appel de **norme**, le compilateur devra incorporer au sein du programme les instructions correspondantes (en langage machine).

La spécification **inline**

Remarques

- A noter qu'une fonction en ligne doit être définie dans le même fichier source que celui où on l'utilise.
- Elle ne peut plus être compilée séparément.
- Pour qu'une même fonction en ligne puisse être partagée par différents programmes, il faudra absolument la placer dans un fichier en-tête.

La spécification **inline**

	Avantages	Inconvénients
Macro	Économie de temps d'exécution	Perte d'espace mémoire Risque d'effets de bord non désirés Pas de compilation séparée possible
Fonction	Économie d'espace mémoire Compilation séparée possible	Perte de temps d'exécution
Fonction en ligne	Économie de temps d'exécution	Perte d'espace mémoire Pas de compilation séparée possible

Le type `bool`

- Le type `bool` est un type forme de deux valeurs `true` ou `false`.
- Théoriquement, les résultats des comparaisons ou des combinaisons logiques doivent être de type `bool`.
- Toutefois, il existe des conversions implicites :
 - de `bool` en numérique
 - `true` devenant `1` et `false` devenant `0`.
 - de numérique (y compris flottant) en `bool`
 - toute valeur non nulle devenant `true` et zero devenant `false`.

Notion d'espace de noms

- Lors de la création de gros programmes ou de bibliothèques, on peut avoir envie de regrouper les symboles (noms de fonctions/de classes/de variables globales) sous une même entité, un nom, qui permettrait de qualifier et de regrouper ces symboles.
- On va alors utiliser des espaces de noms. Nous en avons déjà rencontré un lors de ce cours, que nous avons utilisé. Il s'agissait de l'espace de nom de la bibliothèque standard **std**.

Notion d'espace de noms

Pour déclarer un nouvel espace de nom, on va utiliser le mot clé `<< namespace >>` suivi d'un bloc d'accolades ouvrantes et fermantes. Dans ce bloc, on pourra déclarer des classes, des fonctions, des variables. Dans le fichier de code **cpp** correspondant, on fait la même chose pour la définition des objets déclarés.

```
#ifndef __MYNAMESPACE__
#define __MYNAMESPACE__
namespace MyName{
class A{
/* déclarations de la classe A*/
};
class B{
/* déclarations de la classe B*/
public:
B();
};
void fct (); // une fonction
} // pas de;
#endif
###MyNamespace.h###
```

```
#include "MyNamespace.h"
namespace MyName{
void fct()
{
// instructions
}
B::B()
{
//constructeur de B.
}
}
###MyNamespace.cpp###
```

Notion d'espace de noms

- Pour utiliser les symboles déclarés dans un espace de nom, on va utiliser l'opérateur de résolution de portée <<::>>. Pas de surprise ici, donc.

```
#ifndef __MYNAMESPACE__
#define __MYNAMESPACE__
namespace MyName{
class A{
/* déclarations de la classe A*/
};
class B{
/* déclarations de la classe B*/
public:
B();
};
void fct (); // une fonction
} // pas de;
#endif
###MyNamespace.h###
```

```
#include "MyNamespace.h"
namespace MyName{
void fct()
{
// instructions
}
B::B(){
//constructeur de B.}
}
#include "MyNamespace.h"
int main(){
MyName::fct();
MyName::A a;
}
###MyNamespace.cpp###
```

- On peut également créer un espace de nom, au fur et à mesure. C'est à dire qu'on peut déclarer des symboles comme faisant parti d'un espace de nom dans plusieurs fichiers entête.

La directive `using`

On peut utiliser la directive `using` que nous connaissons déjà, de deux façons.

- La première est de l'utiliser directement avec un espace de nom. Nous reviendrons sur cette utilisation plus tard.
- On peut aussi utiliser `<< using >>` symbole par symbole. Ici, `A` devient synonyme de `MyName A` dans la suite du code.

La directive `using`

```
#ifndef __MYNAMESPACE__
#define __MYNAMESPACE__
namespace MyName{
class A{
/* déclarations de la classe A*/
};
class B{
/* déclarations de la classe B*/
public:
B();
};
void fct (); // une fonction
} // pas de;
#endif
###MyNamespace.h###
```

```
#include "MyNamespace.h"
class A{
public:
double x;
};
int main(){
MyName::fct();
// A est synonyme de
//MyName::A
using MyName::A; ←
A a ;
//a.x = 0; // NE COMPILE PAS
// l'espace de nom a masqué la classe
// A de l'espace global.
// pour atteindre l'espace global.
::A b;
b.x = 0;}
###MyNamespace.cpp###
```

La directive `using`

Deux points sont à noter

- On n'est pas obligés de placer `using` à l'extérieur d'une fonction. Il peut être placé n'importe où dans le code, et n'a d'effet que sur la portée locale c'est à dire jusqu'à la fin du bloc d'instruction courant.
- Dans cet exemple, une classe `A` apparaît dans le namespace `MyName`. Mais une autre classe `A` est déclarée dans le namespace global.

Comment lever l'ambiguïté?

Ici, la directive `using` masque le symbole globale `A`. Pour y accéder quand même, on va utiliser l'opérateur de résolution de portée sur l'espace globale c'est à dire sans préciser d'espace de nom. On lève ainsi l'ambiguïté qui avait été levée.

La directive `using`

```
#ifndef __MYNAMESPACE__
#define __MYNAMESPACE__
namespace MyName
{
class A{
/* déclarations de la classe A*/
};
class B{
/* déclarations de la classe B*/
public:
B();
};
void fct (); // une fonction
} // pas de;
#endif
###MyNamespace.h###
```

```
#include "MyNamespace.h"
class A{
public:
double x;
};
int main(){
MyName::fct();
// A est synonyme de
//MyName::A
using MyName::A;
A a ;
//a.x = 0; // ne compile pas
// l'espace de nom a masqué la class
// A de l'espace global.
// pour atteindre l'espace global.
::A b; ⇐
b.x = 0;}
###main.cpp###
```


La directive `using`

- On peut bien sur utiliser la directive comme on l'a déjà fait pour l'espace de nom `std`. Dans cet exemple, cela ne pose pas de problème.
- Mais que se passe t-il dans le cas où un symbole porte le même nom dans l'espace global?.

La directive **using**

```
#ifndef __MYNAMESPACE__
#define __MYNAMESPACE__
namespace MyName
{
class A{
/* déclarations de la classe A*/
};
class B{
/* déclarations de la classe B*/
public:
B();
};
void fct (); // une fonction
} // pas de;
#endif
```

```
#include "MyNamespace.h"
using namespace MyName;
class A{
public:
double x;
};
int main()
{
fct();
B b;
A a; ←
}
```

La directive **using**

Le compilateur ne sait plus de quel objet il s'agit et s'arrête en erreur. Dans ce cas, il y a deux solutions

- Ne pas utiliser la directive **using**.
- Lever les ambiguïtés au cas par cas, en utilisant l'opérateur quand cela est nécessaire.

Cela nécessite évidemment de savoir quel objet on désire utiliser.

⇒ Facile si c'est son propre code, mais plus délicat si on n'est pas créateur du code à maintenir.

La directive **using**

```
main.cpp: Dans la fonction 'int main()':  
main.cpp:16:5:erreur : reference to 'A' is ambiguous  
A a;  
^  
main.cpp:4:7:note : candidats sont : class A  
class A  
^  
In file included from main.cpp:1:0  
MyNamespace.hh:6:11: note : class MyName ::A  
class A  
^
```

Les classes et Objets

Classes et Objets: Introduction

- Les structures sont des types, définis par l'utilisateur qui contiennent à la fois des données, mais aussi des fonctions membres.
- En C++, ces structures sont un cas particulier d'un mécanisme plus général, les Classes.
- Nous entrons donc maintenant dans la partie Programmation Orientée Objets de ce cours.

Classes et Objets:

Introduction

- Pourquoi ne pas simplement travailler sur des structures. Les structures ne permettent pas d'encapsuler leurs membres, données ou fonctions.
- L'encapsulation fait partie intégrante de la POO. Elle permet de masquer certains membres et fonctions membres au monde extérieur.
- Dans l'idéal, on ne devrait jamais accéder aux données d'une classe, mais agir sur ses méthodes (fonctions membres)

Déclaration

- Une classe = famille d'objets ayant même structure et même comportement.
- Une classe regroupe un ensemble d'attributs ou membres, répartis en données et fonctions.
- Les membres peuvent être : **public**, **protected**, **private** (défaut).
- La définition des fonctions peut être effectuée en ligne.

```
class Point {  
  int x, y; // membres données  
  public:  
    // membres fonctions  
    void setPoint (int, int);  
    /* ou setPoint en ligne  
    void setPoint(int a, int b){  
      x = a; y = b;  
    } */  
    void deplace(int, int);  
    void affiche();  
};
```


Déclaration

- Par défaut, les membres des classes sont privés.
- Les mots clés **public** et **private** permettent de modifier les droits d'accès des membres.

```
class Point {  
    // les membres déclarés ici sont privés  
    public:  
    // les membres déclarés ici sont publics  
    Private:  
    // les membres déclarés ici sont privés  
};
```

Définition

- L'opérateur de portée `::` indique la classe à laquelle appartient la méthode.

```
void Point::setPoint(int a, int b) {  
x = a; y = b;  
}  
void Point::deplace(int dx, int dy) {  
x += dx; y += dy;  
}  
void Point::affiche() {  
cout << "x = " << x << " y = " << y << endl;  
}
```

- Pointeur `this`

```
void Point::setPoint(int a, int b) {  
this->x = a; this->y = b;  
}
```

```
#include< iostream >
using namespace std;
void main() {
    Point a, b;
    a.setPoint(1,2);
    b.setPoint(3,-5);
    a.affiche(); // x = 1 y = 2
    b.affiche(); // x = 3 y = -5
    a.deplace(1,1); // x = 2 y = 3
    a.affiche();
}
```

Encapsulation

- Seules les fonctions d'accès et de modification sont rendues publiques.
- Toute donnée est privée ou protégée.
- L'encapsulation peut se pratiquer en donnant à l'utilisateur
 - un fichier entête contenant la déclaration de la classe;
 - un module objet contenant la version compilée du fichier contenant la définition de la classe.

Le projet est composé de trois fichiers:

- Un fichier `point.h` de déclaration de la classe `Point`.
- Un fichier `point.cpp` d'implémentation des méthodes de la classe.
- Un fichier `main.cpp` d'utilisation.

Le fichier d'implémentation est à terme remplacé par un module `point.o`.

Encapsulation

———— fichier point.h ————

```
#ifndef __POINT_H__ // pour éviter l'inclusion multiple
#define __POINT_H__
class Point {
int x, y;
public:
void setPoint(int, int);
void deplace(int, int);
void affiche();
double distance(Point);
};
#endif
```

Encapsulation

——— fichier point.cpp ———

```
#include < iostream.h >
#include " point.h"
#include < cmath >
void Point::setPoint(int a, int b) {
x = a; y = b;
}
void Point::deplace(int dx, int dy) {
x += dx; y += dy;
}
void Point::affiche() {
cout << "x = " << x << " y = " << y << endl;
}
double Point::distance(Point autrePoint) {
intdx = this-> x - autrePoint.x;
intdy = this-> y - autrePoint.y;
return sqrt(dx*dx+dy*dy);
}
```

Encapsulation

——— fichier main.cpp ———

```
#include <iostream>
using namespace std;
#include "point.h"
void main() {
    Point a, b;
    a.setPoint(1,2);
    b.setPoint(3,-5);
    a.affiche(); // x = 1 y = 2
    b.affiche(); // x = 3 y = -5
    a.deplace(1,1);
    a.affiche(); // x = 2 y = 3
    Point C;
    c.setPoint(1,2);
    double d = c.distance(a);
    cout << "distance entre les 2 points a et c : " << d << endl;
}
```

- Un constructeur est une méthode d'initialisation des attributs d'un objet à la création.
- En C++, un constructeur a le nom de la classe, et pas de type de retour.
- Une classe peut avoir plusieurs constructeurs.
- Un constructeur sans arguments est appelé constructeur par défaut.
 - Ce constructeur existe implicitement, s'il est le seul constructeur.
 - La définition d'un deuxième constructeur exige que l'on définisse explicitement le constructeur par défaut si l'on veut s'en servir.

Le constructeur par défaut est utilisé:

- Lors de la définition d'un objet, par `Point a;`
- lors d'une allocation, par `pa = new Point;`

Sauf écriture explicite, le constructeur par défaut n'effectue pas d'action autre que la réservation de place.

Constructeurs

- Exemple: remplacement de la méthode **setPoint** par un constructeur.

```
class Point {  
    int x, y;  
    public:  
    Point(int, int); // constructeur  
    void deplace(int,int);  
    void affiche();  
    double distance(Point);  
};  
// définition du constructeur  
Point::Point(int a, int b) {  
    x = a; y = b;  
}  
void main() {  
    Point a(1,2), b(3,-5);  
}
```

Constructeurs

- Le constructeur **Point** rend opérant le constructeur par défaut s'il n'est pas redéfini.
- Le constructeur est employé en passant les arguments en paramètres. C'est une abréviation de **Point a=Point(1,2), b=Point(3,-5);**
- Un constructeur simple peut être défini en ligne par:

```
class Point {  
    int x,y;  
    public:  
    Point(){} // constructeur par défaut  
    Point(int a, int b) { x =a; y = b;}  
    ...  
};
```

Exemples d'utilisation de la classe Point:

```
void main() {  
    Point p; // par défaut  
    Point q(4,5); // deuxième  
    Point* a = new Point; // par défaut  
    Point* b = new Point(4,5); // deuxième  
    delete a, b; // libération  
}
```

Constructeur de copie

- Le constructeur de copie est utilisé pour créer un objet à partir d'un autre du même type.
- Supposons que chaque **point** est caractérisé, en plus de ses coordonnées **x** et **y**, par une étiquette (**label**).
- La classe **Point** deviendra **PointNomme**.

```
class PointNomme {  
    int x, y;  
    char *label;  
Public:  
    // constructeur par copie  
    PointNomme(PointNomme & p) {  
        x = p.x; y = p.y;  
        label = new char[strlen(p.label)+1];  
        strcpy(label,p.label);  
    }  
}
```

Construction des objets membres, liste d'initialisation

Un constructeur peut recevoir une liste d'initialisations qui sont effectuées à la construction de l'objet.

Ceci est important quand un ou des champs de l'objet sont eux mêmes des objets. La construction d'un objet par un constructeur sans liste d'initialisation se fait en trois étapes:

- réservation de la place pour l'objet;
- construction des champs par appel du constructeur par défaut de chacun des objets;
- exécution du corps de la fonction.

En présence d'une liste d'initialisation, les constructeurs par défaut sont remplacés par les appels spécifiés dans la liste.

Construction des objets membres, liste d'initialisation

Exemple: une classe de points

```
class Point {  
  int x, y;  
  public:  
  Point(int abs = 0, int ord = 0);  
};
```

et une classe de segments, formées de deux points.

```
class Segment {  
  Point d, f;  
  int epaisseur;  
  public:  
  Segment(int dx, int dy, int fx, int fy, int ep);  
};
```

Construction des objets membres, liste d'initialisation

Première implémentation du constructeur

```
Segment::Segment(int dx,int dy, int fx, int fy, int ep) {  
    d = Point(dx,dy); f = Point(fx,fy);  
    epaisseur = ep;  
}
```

La construction

`Segment k = Segment(1,2,3,4,1);`

provoque:

- deux appels au constructeur par défaut `Point()` pour l'initialisation de `d` et de `f`, et ceci avant d'entrer dans le corps du constructeur de segments;
- les appels de constructeurs `Point(1,2)` et `Point(3,4)` pour construire des objets temporaires;
- trois emplois de l'opérateur d'affectation;
- deux appels au destructeur pour détruire les objets temporaires

Construction des objets membres, liste d'initialisation

Deuxième implémentation du constructeur

```
Segment::Segment(int dx,int dy, int fx, int fy, int ep): d(dx,dy), f(fx,fy),  
    epaisseur(ep) {}
```

La construction ne nécessite que deux appels de constructeurs, pour construire **d** à partir de **(dx,dy)** et de **f** à partir de **(fx,fy)**.

Utiliser le plus souvent possible les listes d'initialisation.

Destructeurs

- Un destructeur est une fonction membre spéciale. Il a le même nom que la classe, précédé du caractère `~`.
- Un destructeur n'a pas de paramètres, ni type de retour.
- Il y a au plus un destructeur par classe.
- Le destructeur est utilisé:
 - Lorsque le programme quitte le bloc où l'objet est déclaré;
 - Pour la destruction explicite par `delete pointeur_class`.

```
class Id {  
    string nom;  
public:  
    Id();  
    ~Id();  
};
```

Destructeurs

Le constructeur et le destructeur sont définis comme suit :

```
Id::Id() {  
    cout << " Entrez votre nom : ";  
    cin >> nom;  
}  
Id::~~Id() {  
    cout << " Mon nom est " << nom << endl; }
```

Le programme se réduit à:

```
void main(){  
    Id Mohamed, Ahmed;}
```

Output

```
Entrez votre nom:Mohamed  
Entrez votre nom:Ahmed  
Mon nom est Ahmed  
Mon nom est Mohamed
```

La classe Point

```
class Point{
public:
    Point(); // constructeur par défaut
    Point(int a, int b, char* s ); // constructeur
    // normal
    Point(const Point &autrePoint); // constructeur par copie
    ~Point();
    void affiche();
private:
    int x, y;
    char *nomPoint;
};
```

La classe Point

```
Point::Point(){  
    cout << " Constructeur par default " << this << endl;  
    x=y=0; nomPoint =0;  
}  
Point::Point(int a, int b, char* s ){  
    x=a; y=b;  
    nomPoint = new char[strlen(s)+1];  
    strcpy(nomPoint, s);  
    cout << " Constructeur normal " << this << endl;  
}  
Point::Point(const Point &autrePoint){  
    x = autrePoint.x; y = autrePoint.y;  
    nomPoint = new char[strlen(autrePoint.nomPoint)+1];  
    strcpy(nomPoint, autrePoint.nomPoint);  
    cout << " Constructeur par copie " << this << endl;  
}
```

La classe Point

```
Point::~~Point(){
delete [] nomPoint;
cout << " Destructeur " << this << endl;
}
void main(){
Point a; // par défaut
Point b(1,2,"Point_1"); // normal
Point * c = new Point; // par défaut
Point * d = new Point(3,4, "Point_2"); // normal
Point e(b); // par copie
delete c;
delete d;
}
```

Résultat d'exécution :

Constructeur par défaut 0012FF68

Constructeur normal 0012FF5C

Constructeur par défaut 00481F20

Constructeur normal 00481EE0

Constructeur par copie 0012FF48

Destructeur 00481F20

Destructeur 00481EE0

Destructeur 0012FF48

Destructeur 0012FF5C

Destructeur 0012FF68

Membres constants

Données membres constantes

- Une donnée membre d'une classe peut être qualifiée **const**.
- Il est obligatoire de l'initialiser lors de la construction d'un objet.
- Sa valeur ne pourra par la suite être modifiée.

Exemple: nouvelle version de la classe **Segment**

```
class Segment {  
    Point origine, extremite;  
    int epaisseur;  
    const int numeroDeSerie;  
public:  
    Segment(int x1, int y1, int x2, int y2, int ep, int num);  
};
```

Données membres constantes

Constructeur, version erronée:

```
Segment::Segment(int x1, int y1, int x2, int y2, int  
ep, int num) :origine(x1,y1),extremite(x2,y2) {  
    epaisseur = ep;  
    numeroDeSerie = num; // ERREUR: tentative de modification d'une constante  
}
```

Constructeur, version correcte, utilisation de la syntaxe de l'initialisation des objets membres:

```
Segment::Segment(int x1, int y1, int x2, int y2, int  
ep, int num) :origine(x1,y1),extremite(x2,y2),  
    numeroDeSerie(num) {  
    epaisseur = ep;  
}
```


Fonctions membres constantes

Une fonction membre constante est une fonction qui ne modifie pas l'objet appelant.

```
class Point {  
    ....  
    void placer(int a, int b); // modifie l'objet  
    float distance(Point p) const; // ne modifie pas l'objet  
    ...  
}
```

A l'intérieur d'une fonction **const** d'une classe C le pointeur this est de type "const C* const".

```
void uneFonction(const Point a){  
    Point b;  
    ...  
    float d = a.distance(b);  
    ...  
}
```

Membres statiques

- Les membres statiques sont partagés par tous les objets de la classe.
- Un membre statique est unique pour toute la classe quel que soit le nombre d'objets de la classe.
- Les données et fonctions membres non statiques sont appelées **variables d'instance** et **méthodes d'instance**, respectivement.
- Les données et fonctions membres statiques sont appelées **variables de classe** et **méthodes de classe**, respectivement.

Une donnée membre statique est spécifiée par **static**.

- Elle n'est instanciée qu'une seule fois;
- Elle est commune à toutes les instances de la classe;
- Si un objet la modifie, elle est modifiée pour tous les objets.
- Elle est initialisée avant utilisation.

Une donnée statique et constante est une donnée immuable.

Données membres statiques

```
class Point {  
    int x, y;  
public:  
    static int nombreDePoints;  
    Point(int a, int b) {  
        x = a; y = b;  
        nombreDePoints++;  
    }  
};
```

- Il faut créer et initialiser la donnée membre **nombreDePoints**.
- Pour une donnée membre non statique cela est fait par le constructeur.

```
int Point::nombreDePoints = 0;
```

- La ligne ci-dessus doit être écrite dans un fichier **.cpp**.

Données membres statiques

- L'accès à un membre statique depuis une fonction non membre peut se faire à travers n'importe quel objet de la classe:

```
Point a, b,c;
```

```
...
```

```
cout << a.nombreDePoints << endl;
```

- L'accès peut s'écrire indépendamment de tout objet:

```
cout << Point :: nombreDePoints << endl;
```

Fonctions membre statiques

Un méthode statique est spécifiée par `static`.

- Elle ne fait pas référence à un objet (pas de `this`);
- Elle est appelée comme une fonction globale;
- S'il faut préciser sa classe, son nom est préfixé à l'aide de l'opérateur de portée : `Math::pgcd()`

```
class Point {  
    int x, y;  
    static int nombreDePoints; //Private  
    public:  
    static int combien(){//pour consulter la valeur de nombreDePoints  
        return nombreDePoints;  
    }  
    Point(int a, int b) {  
        x = a; y = b;  
        nombreDePoints++;  
    }  
};
```

Fonctions membre statiques

Pour afficher le nombre de points on devra écrire une expression comme (**a** étant de type **Point**) :

```
cout << a.combien() << endl;
```

- ou encore mieux, une expression ne fait pas intervenir de point particulier:

```
cout << Point :: combien() << endl;
```

Le mot clé this

Chaque fonction membre d'un objet reçoit une information supplémentaire. Elle permet de faire le lien entre les corps des fonctions membres et l'instance courante de la classe. Il s'agit de **this**. C'est un pointeur transmis à toutes les fonctions membres qui pointe vers l'instance courante.

Le mot clé this

```
#include < iostream >
class Objet{
public:
    Objet();
    ~Objet();
};
Objet::Objet(){
    std :: cout << " C : " << this << std :: endl;
}
Objet::~~Objet(){ std :: cout << " D : " << this << std :: endl;
}
int main(){ Objet o, op;}
```

Sortie

```
$ ./a.exe
C : 0xffffcbff
C : 0xffffcbfe
D : 0xffffcbfe
D : 0xffffcbff
```


Accesseurs et Mutateurs

```
#ifndef __OBJET_H__
#define __OBJET_H__
class Objet{
private:
double _v;
public:
Objet(double _v);
~Objet();
double getV () const;
void setV(double d);
};
#endif
#####Objet.h#####
```

```
#include "Objet.h"
Objet::Objet( double v )
: _v(v){}
Objet::Objet (){}
double Objet::getV ()
const
{
return _v;
}
void Objet::setV(double
v)
{
_v = v;
}
#####Objet.cpp#####
```

```
#include < iostream >
#include "Objet.h"
using namespace std;
int main()
{
Objet o(3);
o.setV(4);
cout << o.getV() <<
endl;
}
#####main.cpp#####
said@win~/setter
$ g++ main.cpp
Objet.cpp
said@win~/setter
$ ./a.exe
4
```

Accesseurs et Mutateurs

```
#ifndef __OBJET_H__
#define __OBJET_H__
class Objet{
private:
double _v;
public:
Objet(double _v);
~Objet();
double getV () const;
void setV(double d);
};
#endif
####Objet.h####
```

```
#include "Objet.h"
Objet::Objet( double v )
: _v(v){}
Objet::Objet (){}
double Objet::getV ()
const
{
return _v;
}
void Objet::setV(double
v)
{
_v = v;
}
####Objet.cpp####
```

On appelle accesseurs et mutateurs des fonctions permettant l'accès à des attributs privés d'une classe. On les appelle aussi getter et setter en anglais. Ce sont des fonctions qui doivent être presque automatiquement créées lors de la création

Accesseurs et Mutateurs

```
#ifndef __OBJET_H__
#define __OBJET_H__
class Objet{
private:
double _v;
public:
Objet(double _v);
~Objet();
double getV () const;
void setV(double d);
};
#endif
####Objet.h####
```

```
#include "Objet.h"
Objet::Objet( double v )
: _v(v){}
Objet::Objet (){}
double Objet::getV ()
const
{
return _v;
}
void Objet::setV(double
v)
{
_v = v;
}
####Objet.cpp####
```

Leurs noms rappellent les noms des attributs précédés de get pour les getters et set pour les setters. En général, on déclare les fonctions getters comme étant **const** comme dans l'exemple ci-contre.

Accesseurs et Mutateurs

```
#ifndef __OBJET_H__
#define __OBJET_H__
class Objet{
private:
double _v;
public:
Objet(double _v);
~Objet();
double getV () const;
void setV(double d);
};
#endif
#####Objet.h#####
```

```
#include "Objet.h"
Objet::Objet( double v )
: _v(v){}
Objet::Objet (){}
double Objet::getV ()
const
{
return _v;
}
void Objet::setV(double
v)
{
_v = v;
}
#####Objet.cpp#####
```

En effet, les fonctions membres déclarés comme **const** permettent à l'utilisateur de cette méthode de savoir que cette fonction ne modifiera pas les champs de l'objet. Ce sont aussi les seuls fonctions que l'on peut appeler sur des objets constants.

Accesseurs et Mutateurs

```
#ifndef __OBJET_H__
#define __OBJET_H__
class Objet{
private:
double _v;
public:
Objet(double _v);
~Objet();
double getV () const;
void setV(double d);
};
#endif
####Objet.h####
```

```
#include "Objet.h"
Objet::Objet( double v )
: _v(v){}
Objet::Objet (){}
double Objet::getV ()
const
{
return _v;
}
void Objet::setV(double
v)
{
_v = v;
}
####Objet.cpp####
```

On pourra noter également la syntaxe du constructeur qui initialise le champ `_v` de l'instance en lui passant la valeur entre parenthèse, comme dans le cas des objets imbriqués. Il est néanmoins nécessaire d'ajouter les accolades, même si le corps est vide.

- Dans une classe, la déclaration **friend** permet d'accorder l'accès aux membres privés ou protégés à des fonctions isolées, ou à toutes les fonctions d'une classe.
- Cela sert notamment quand une opération utilise des données de deux ou plusieurs classes

Différentes situations d'amitié

- Fonction indépendante, amie d'une classe;
- Fonction membre d'une classe, amie d'une autre classe;
- Fonction amie de plusieurs classes;
- Toutes les fonctions membres d'une classe, amies d'autre classe.

Fonction indépendante, amie d'une classe

```
class Point{  
    int x, y;  
    public:  
    Point(int abs=0, int ord=0) {  
        // constructeur inline  
        x = abs; y = ord;  
    } // déclaration de fonction amie (indépendante)  
    friend coincide(Point p, Point q);  
};
```

```
int coincide (point p, point q){  
    // définition de la fonction amie (indépendante)  
    if ((p.x==q.x)&&(p.y==q.y)) return 1; else return 0;  
};
```

Fonction indépendante, amie d'une classe

- Utilisation dans un programme

```
main(){  
  // utilisation de la fonction amie (indépendante)  
  Point a(1,0), b(1), c;  
  if (coincide(a,b)) cout << "a coincide avec b \ n"; else  
  cout << "a et b sont differents \ n"; if (coincide(a,c))  
  cout << "a coincide avec c \ n"; else cout << "a et c sont differents \ n"; };
```

- Sortie du programme

a coincide avec b

a et c sont différents

Fonction membre d'une classe, amie d'une autre classe

```
class A;  
class B {  
int f(char , A);  
...  
};  
class A {  
...  
friend int B::f(char , A);  
...  
};  
int B::f(char ..., A ...) {  
...  
}
```

Fonction membre d'une classe, amie d'une autre classe

```
class vect; // pour pouvoir compiler correctement
class matrice { // déclaration de la classe matrice
double mat[3][3]; // matrice 3 × 3
public:
matrice (double t[3][3]){// constructeur
int i, j;
for(i=0;i<3;i++)
for(j=0;j<3;j++)
mat[i][j] = t[i][j];
}
vect prod(vect); // fonction membre
};
class vect { // déclaration de la classe vect
double v[3];
public :
vect (double v1=0,double v2=0,double v3=0){//constructeur
v[0]=v1;v[1]=v2;v[2]=v3;
}
}
friend vect matrice::prod(vect);
```

Fonction membre d'une classe, amie d'une autre classe

```
void affiche(){
int i;
for(i=0;i<3;i++) cout << v[i] << " ";
cout << endl;
}
}; // fin de déclaration de la classe vect
// définition de la fonction prod
vect matrice::prod(vect x){
int i,j; double som;
vect res; //pour le résultat
for (i=0;i<3;i++) {
som = 0;
for (j=0;j<3;j++) som += mat[i][j]*x.v[j];
res.v[i] = som;
}
return res;
}
main(){
vect w(1,2,3); vect res;
double tb[3][3]={1,2,3,4,5,6,7,8,9};
matrice a = tb;
res= a.prod(w); res.affiche();
}
```

14 32 50

Fonction amie de plusieurs classes

```
class B;
class A {
...
friend void f(A,B);
...
};
class B {
...
friend void f(A,B);
...
};
void f(A...,B...) {
// on a accès ici aux membres privés de
// n'importe quel objet de type A ou B
}
```

Classe amie (toutes les fonctions d'une classe sont amies d'une autre classe)

```
class B; // nécessaire pour la compilation de la classe A
class A { // déclaration de la classe A
...
friend class B; // toutes les fonctions membres de la
//classe B sont amies de la classe B
...
};
class B { // déclaration de la classe B
...
};
```

La Surdéfinition des Opérateurs

Introduction

- C++ autorise la surdéfinition (surcharge) de fonctions (membres ou indépendantes).
- Selon le type de **a** et **b**, dans l'expression **a+b** le symbole **+** peut designer:
 - l'addition de deux entiers ;
 - l'addition de deux réels (**float**);
 - etc
- Supposons que nous nous disposons d'une classe **Complexe** destinée à représenter les nombres complexes.
 - Si **a** et **b** sont deux objets de type **Complexe**, les expressions **a+b** **a-b** **a*b** **a/b** sont elles acceptées en C++?
 - Non!
 - Il faut surdéfinir les opérateurs **+**, **-**, ***** et **/** en spécifiant le rôle exact que vous souhaitez leur attribuer.

Le mécanisme de la surdéfinition d'opérateurs

Considérons une classe point :

```
class point {  
  int x, y ;  
  .....  
};
```

- La somme de deux points est un point dont les coordonnées sont la somme de leurs coordonnées.
- Nous souhaitons utiliser l'expression $a + b$ (a et b de type `point`) pour calculer la somme de a et b .
- Il faut surdéfinir (surcharger) l'opérateur $+$. Comment?
 - Définir une fonction de nom :
`operator+`
 - Le mot clé `operator` est suivi de l'opérateur à surdéfinir (dans notre cas $+$).
 - L'espace entre `operator` et $+$ est optionnel (pas obligatoire).

Le mécanisme de la surdéfinition d'opérateurs

- la fonction `operator+` a pour objectif de calculer la somme de deux points.
 - elle doit disposer de deux arguments de type `point` et fournir une valeur de retour du même type (`point`).

Deux possibilités offertes pour la mise en œuvre de la surdéfinition:

- 1 la fonction `operator+` peut être une fonction membre de la classe concernée.
- 2 ou une fonction indépendante (fonction amie).

Surdéfinition d'opérateur avec une fonction amie

- Le prototype de la fonction `operator +` sera :
`point operator + (point, point) ;`
- Pour la mise en oeuvre de la surdéfinition de l'opérateur `+`
 - il faut déclarer l'amitié au sein de la classe `point` ;

```
class point{  
    int x, y ;  
    public :  
    friend point operator+ (point, point) ;  
    ...  
} ;
```

- définir la fonction `operator+` a l'extérieur de la classe `point`.

```
point operator + (point a, point b){  
    point p ;  
    p.x = a.x + b.x ; p.y = a.y + b.y ;  
    return p ;  
}
```

Surdéfinition d'opérateur avec une fonction amie: Exemple 1

```
class point {  
    int x, y ;  
    public :  
    point (int abs=0, int ord=0) { x=abs ; y=ord ;} // constructeur  
    friend point operator+ (point, point) ;  
    void affiche () { cout << "coordonnees : " << x << " " << y << " \n"; }  
};  
  
point operator + (point a, point b){  
    point p ;  
    p.x = a.x + b.x ; p.y = a.y + b.y ;  
    return p ;  
}  
  
main(){  
    point a(1,2) ; a.affiche() ;  
    point b(2,5) ; b.affiche() ;  
    point c ;  
    c = a+b ; c.affiche() ;  
    c = a+b+c ; c.affiche() ;}
```

Surdéfinition d'opérateur avec une fonction amie:

Exemple 1

Output

```
coordonnees : 1 2  
coordonnees : 2 5  
coordonnees : 3 7  
coordonnees : 6 14
```

Surdéfinition d'opérateur avec une fonction amie:

Exemple 1

- $c = a + b$; est équivalente à $c = \text{operator} + (a, b)$;
- Associativité:
 $(a + b) + c$ est la même chose que
 $\text{operator} + (\text{operator} + (a, b), c)$

Surdéfinition d'opérateur avec une fonction membre

- Le premier argument (premier opérande de notre opérateur `+`) de la fonction `operator +` va se trouver transmis implicitement : ce sera l'objet ayant appelé la fonction membre.
- Une expression telle que:
`a + b`
sera interprétée par le compilateur comme:
`a.operator + (b)`
- Le prototype de la fonction membre `operator +` sera donc :
`point operator + (point)`

Surdéfinition d'opérateur avec une fonction membre: Exemple 2

L'exemple-1 deviendra:

```
class point {  
    int x, y ;  
    public :  
    point (int abs=0, int ord=0) { // constructeur  
        x=abs ; y=ord ;  
    }  
    point operator + (point) ;  
    void affiche () {  
        cout << "coordonnees : " << x << " " << y << " \n";  
    }  
};  
point point::operator + (point a)  
{  
    return point(x + a.x , y + a.y ) ;  
}
```

Surdéfinition d'opérateur avec une fonction membre:

Exemple 2

```
main()  
{ point a(1,2) ; a.affiche() ;  
  point b(2,5) ; b.affiche() ;  
  point c ;  
  c = a+b ; c.affiche() ;  
  c = a+b+c ; c.affiche() ;  
}
```

```
coordonnees : 1 2  
coordonnees : 2 5  
coordonnees : 3 7  
coordonnees : 6 14
```


Surdéfinition d'opérateur avec une fonction membre:

Exemple 2

```
point point::operator + (point a)
{
    point p ;
    p.x = x + a.x ; p.y = y + a.y ;
    return p ;
}
```

Remarques:

- 1 Dissymétrie entre les deux opérandes dans la définition de la fonction `operator +`:
 - `x` pour le premier opérande (argument implicite)
 - `a.x` pour le second opérande

- 2 L'affectation :
`c = a + b;`
est interprétée comme :
`c = a.operator + (b);`

- 3 L'interprétation de l'affectation :
`c = a + b + c;`

dépend du compilateur utilisé:

- Soit créer un objet temporaire `t`:
`t = a.operator + (b);`
`c = t.operator + (c) ;`
- ou `c = (a.operator + (b)).operator + (c)`

Opérateurs et transmission par référence

- Dans le cas d'objets de grande taille, il est intéressant de faire appel au transfert par référence pour rendre le programme rapide:
 - fonction amie :
`point operator + (point & a, point & b) ;`
 - fonction membre :
`point operator + (point & a) ;`
- Afin d'empêcher la modification des arguments passés en référence, on utilise (la forme la plus conseillée):
 - fonction amie :
`point operator + (const point & a, const point & b) ;`
 - fonction membre :
`point operator + (const point & a) ;`

Surdéfinition des opérateurs ++ et --

- On peut définir a la fois un opérateur ++ utilisable en notation préfixée, et un autre utilisable en notation postfixée.
- Une convention qui consiste a ajouter un argument fictif supplémentaire a la version postfixée.
- Surdéfinition de ++ avec une fonction membre (T désigne une classe:
 - Notation préfixée:
T operator ++ ()
 - Notation postfixée:
T operator ++ (int)L'argument de type int est facultatif (afin de distinguer post- et pré-incrementation).

Surdéfinition des opérateurs `++` et `--`

- Surdéfinition de `++` avec une fonction non membre (fonction amie), `T` désigne une classe:
 - Notation préfixée:
`T operator (T)`
 - Notation postfixée:
`T operator (T, int)`
- Même chose pour l'opérateur `--`.

Surdéfinition des opérateurs ++ et --

Exemple

```
class point{
int x, y ;
public :
point (int abs=0, int ord=0) { x=abs ; y=ord ; }
point operator ++ () // notation prefixée
{ x++ ; y++ ; return *this ; }
point operator ++ (int n) // notation postfixée
{
point p = *this ; x++ ; y++ ;
return p ;
}
void affiche () {cout << x << " " << y << " \n"; }
};

main(){
point a1 (2, 5), a2(2, 5), b ;
b = ++a1 ; cout << "a1 : " ; a1.affiche () ; // affiche a1 : 3 6
cout << "b : " ; b.affiche () ; // affiche b : 3 6
b = a2++ ; cout << "a2 : " ; a2.affiche () ; // affiche a2 : 3 6
cout << "b : " ; b.affiche () ; // affiche b : 2 5
}
```

L'opérateur `=` et le constructeur de copie

- En l'absence de surdéfinition explicite, l'opérateur `=` correspond la copie des valeurs de son second opérande dans le premier.
`A=B; //A et B deux objets du même type`
- Cette simple copie peut causer des problèmes lorsque les objets concernés comportaient des pointeurs sur des emplacements dynamiques.
- Il y a des situations qui nécessitent la surdéfinition de l'opérateur `=`.
- Il faut noter que:
 - le constructeur de copie : s'il n'en existe pas d'explicite, il y a appel d'un constructeur de copie par défaut ;
 - l'opérateur d'affectation : s'il n'en existe pas d'explicite, il y a emploi d'un opérateur d'affectation par défaut.
- Constructeur de copie par défaut et opérateur d'affectation par défaut effectuent le même travail : la copie des valeurs de l'objet.

Problème lié au constructeur de copie par défaut

- Considérons la classe:

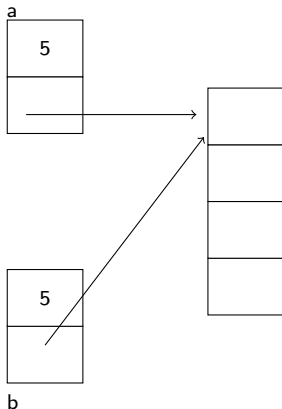
```
class vect {  
    int nelem ; // nombre d'elements  
    int * adr ; // adresse  
    public :  
    vect (int n) // constructeur  
    .....  
} ;
```

- Si `fct` est une fonction a un argument de type `vect`, les instructions
`vect a(5);`

```
...  
fct(a);  
posent problème!
```

Problème lié au constructeur de recopie par défaut

- L'appel de **fct** conduisait à la création, par recopie de **a**, d'un nouvel objet **b**.
- Présence de deux objets **a** et **b** comportant un pointeur (**adr**) vers le même emplacement :

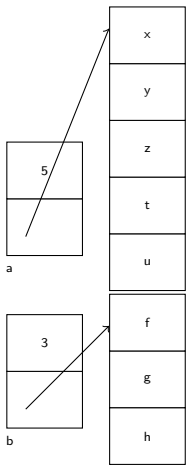


- Problème: Si la classe **vect** possédait un destructeur, on risquait d'aboutir à deux demandes de libération du même emplacement mémoire.
- Solution: Une solution consistait à définir un constructeur de recopie chargé d'effectuer non seulement la recopie de l'objet lui-même, mais aussi celle de sa partie dynamique dans un nouvel emplacement.

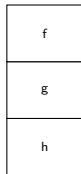
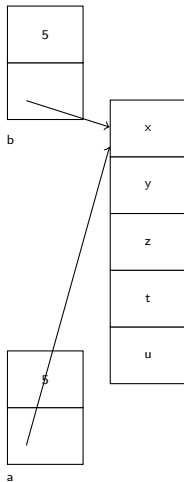
Problème de l'affectation par défaut

- L'affectation d'objets de type **vect** pose les mêmes problèmes.

- Ainsi, avec la déclaration :
vect a(5), b(3); on aura



- L'affectation
b=a; conduit a:

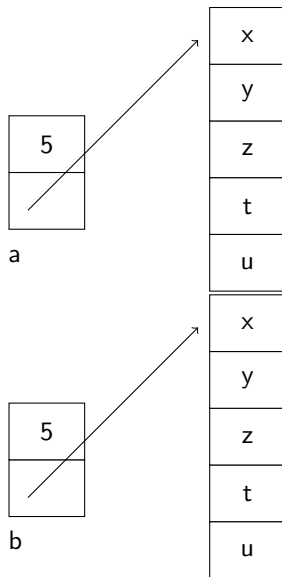


- Problème d'utilisation de l'opérateur **=** par défaut.
- Il faut surdéfinir l'opérateur **=**.

Surdéfinition de $=$

Si a est différent de b , l'affectation $b = a$ peut se traiter comme suit:

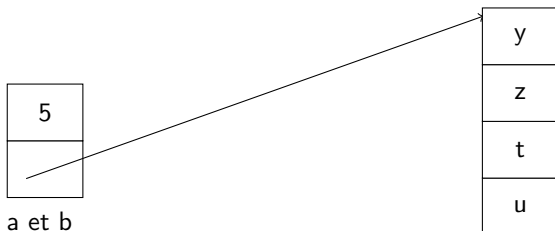
- libération de l'emplacement pointé par b ;
- création dynamique d'un nouvel emplacement dans lequel on recopie les valeurs de l'emplacement pointé par a ;
- mise en place des valeurs des membres données de b .



Surdéfinition de $\equiv$

Si **a** et **b** correspondent au même objet:

- Si la transmission de **a** à l'opérateur d'affectation a lieu par valeur, et si le constructeur par recopie a été redéfini de façon appropriée, l'algorithme proposé fonctionnera sans problème.
- Si la transmission de **a** a lieu par référence, l'emplacement dynamique associé à **b** (donc aussi à **a**) sera libéré avant qu'on tente de l'utiliser pour le recopier dans un nouvel emplacement. La situation sera alors catastrophique.



Surdéfinition de `=`

Valeur de retour:

- Si nous nous contentons d'affectations simples (`b=a`), nous n'avons besoin d'aucune valeur de retour (`void`).
- Si nous souhaitons pouvoir traiter une affectation multiple, il est nécessaire que l'opérateur fournisse une valeur de retour.

Surdéfinition de `=`

Solution proposée (fonction membre):

- `b` devient le premier opérande ici `this`
- et `a` devient le second opérande ici `v`

```
vect & vect::operator = (const vect & v) // notez const
{
    if (this != &v)
    { delete adr ;
      adr = new int [nelem = v.nelem] ;
      for (int i=0 ; i<nelem ; i++) adr[i] = v.adr[i] ;
    }
    return * this ;
}
```

Surdéfinition de =

Exemple complet - Déclaration de la classe **vect**

```
//—— Déclaration de la classe vect ——  
class vect {  
    int nelem ; // nombre d'elements  
    int * adr ; // pointeur sur ces elements  
public :  
    vect (int n) // constructeur  
    {  
        adr = new int [nelem = n] ;  
        for (int i=0 ; i<nelem ; i++) adr[i] = 0 ;  
        cout << " ++ obj taille " << nelem << " en " << this  
        << " - v. dyn en " << adr << " \ n";  
    }  
    ~vect () // destructeur  
    {  
        cout << " -- obj taille " << nelem << " en "  
        << this << " - v. dyn en " << adr << " \ n";  
        delete adr ;  
    }  
}
```

Surdéfinition de ==

Exemple complet – Définition de la fonction operator==

//—— Définition de la fonction **operator==** ——

```
vect & vect::operator == (const vect & v)
{
    cout << " == appel operateur = avec adresses " << this << " " << &v <<
    " \n";
    if (this != &v)
    {
        cout << " effacement vecteur dynamique en " << adr << " \n";
        delete adr ;
        adr = new int [nelem = v.nelem] ;
        cout << " nouveau vecteur dynamique en " << adr << " \n";
        for (int i=0 ; i<nelem ; i++) adr[i] = v.adr[i] ;
    }
    else cout << " on ne fait rien \n";
    return * this ;
}
```

Surdéfinition de =

Exemple complet – Prog. principal

```
//—— Prog. principal ——  
main()  
{  
  vect a(5), b(3), c(4) ;  
  const vect d(1);  
  a = b;  
  cout << " ** affectation a = b \ n";  
  a = b ;  
  cout << " ** affectation c = c \ n";  
  c = c ;  
  cout << " ** affectation a = b = c \ n";  
  a = b = c ;  
}
```


Surdéfinition de =

Exemple complet – Sortie (1/2)

```
++ obj taille 5 en 0xbf8614d0 - v. dyn en 0x9bf9008
++ obj taille 3 en 0xbf8614d8 - v. dyn en 0x9bf9020
++ obj taille 4 en 0xbf8614e0 - v. dyn en 0x9bf9030
++ obj taille 1 en 0xbf8614e8 - v. dyn en 0x9bf9048
== appel operateur = avec adresses 0xbf8614d0 0xbf8614e8
effacement vecteur dynamique en 0x9bf9008
nouveau vecteur dynamique en 0x9bf9058
** affectation a=b
== appel operateur = avec adresses 0xbf8614d0 0xbf8614d8
effacement vecteur dynamique en 0x9bf9058
nouveau vecteur dynamique en 0x9bf9058
** affectation c=c
== appel operateur = avec adresses 0xbf8614e0 0xbf8614e0
on ne fait rien
```

Surdéfinition de ==

Exemple complet – Sortie (2/2)

**** affectation a=b=c**

== appel operateur == avec adresses 0xbf8614d8 0xbf8614e0

effacement vecteur dynamique en 0x9bf9020

nouveau vecteur dynamique en 0x9bf9008

== appel operateur == avec adresses 0xbf8614d0 0xbf8614d8

effacement vecteur dynamique en 0x9bf9058

nouveau vecteur dynamique en 0x9bf9068

– obj taille 1 en 0xbf8614e8 - v. dyn en 0x9bf9048

– obj taille 4 en 0xbf8614e0 - v. dyn en 0x9bf9030

– obj taille 4 en 0xbf8614d8 - v. dyn en 0x9bf9008

– obj taille 4 en 0xbf8614d0 - v. dyn en 0x9bf9068

La forme canonique d'une classe

- Lorsque une classe dispose des parties dynamiques, la copie des objets (constructeur de recopie ou opérateur d'affectation) de la classe ne fonctionne pas correctement!
- Afin que le recopie fonctionne convenablement, il faut mener la classe de :
 - constructeur (s'occupe de l'allocation des parties dynamiques de l'objet) ;
 - destructeur (libérer les emplacements dynamiques créés par l'objet) ;
 - constructeur de recopie ;
 - opérateur d'affectation.

```
class T {  
public :  
    T (...) ; // constructeurs autres que par recopie  
    T (const T &) ; // constructeur de recopie (forme conseillée)  
    // (d'éclaration privée pour l'interdire)  
    ~T () ; // destructeur  
    T & operator = (const T &) ; // affectation (forme conseillée)  
    // (déclaration privée pour l'interdire)  
    ...  
};
```

Surdéfinition de l'opérateur []

Exercice

```
class vect {  
  int nelem ;  
  int * adr ;  
  ....  
};
```

Surcharger l'opérateur [] afin qu'on puisse l'utiliser pour :

- consulter les éléments d'un objet **a** en utilisant **a[i]** (l'utiliser dans une expression)
- pouvoir l'utiliser a gauche d'une affectation (lvalue), c-a-d,
a[i] =

les patrons de fonctions et classes

Patrons de fonctions : Création d'un patron de fonctions

- Comment on peut écrire une fonction **min** fournissant le minimum de deux valeurs de même type reçues en arguments?

- Définition de la fonction **min** pour le type **int**:

```
int min (int a, int b)
{
    if (a < b) return a ; // ou return a < b ? a : b ;
    else return b ;
}
```

- Définition de la même fonction pour le type **float**

```
float min (float a, float b)
{
    if (a < b) return a ; // ou return a < b ? a : b ;
    else return b ;
}
```

Création d'un patron de fonctions

- Besoin d'autres fonctions pour d'autres types.
 - Il s'agit d'écrire nombreuses fonctions!!
- Solution: utilisation d'un seul patron de fonctions:

```
template <class T> T min (T a, T b)
{
if (a < b) return a ; // ou return a < b ? a : b ;
else return b ;
}
```

- L'en-tête de la fonction est devenu:

```
template <class T> T min (T a, T b)
```

- `template <class T>` précise que l'on a affaire à un patron (`template`) dans lequel apparaît un paramètre de type nommé `T`.
- Le mot-clé `class` est utilisé pour préciser que `T` est un paramètre de type.
- `T min (T a, T b)` précise que `min` est une fonction recevant deux arguments de type `T` et fournissant un résultat du même type.

Utilisations du patron de fonctions

```
// Création d'un patron de fonctions
template <class T > T min (T a, T b)
{ if (a < b) return a ; // ou return a < b ? a : b ;
  else return b ;
}
// Exemple d'utilisation du patron de fonctions min
main()
{ int n=4, p=12 ;
  float x=2.5, y=3.25 ;
  cout << "min(n, p) = " << min(n, p) << " \ n";
  cout << "min(x, y) = " << min(x, y) << " \ n";
}
```

```
min (n, p) = 4
min (x, y) = 2.5
```

- Avec l'appel **min (n, p)**, le compilateur fabriquera (instanciera) la fonction **min** (dite fonction patron) correspondant à des arguments de type **int**.
- Même chose pour **min** avec deux arguments de type **float**, le compilateur instanciera une autre fonction patron **min** correspondant a des arguments de type **float**, et ainsi de suite.

Utilisations du patron de fonctions

- Le patron **min** peut être utilisé pour des arguments de n'importe quel type:
 - Type prédefini (short, char, double, int *, char *, int **, etc.)
 - Type défini par l'utilisateur (notamment structure ou classe).
- Si **n** et **p** sont de type **int**, un appel tel que **min (&n, &p)** conduit le compilateur à instancier une fonction **int * min (int *, int *)**.
Le code:

```
int n=4, p=12 ;  
cout << "&n = " << &n << " &p = " << &p << " \ n";  
cout << " min(&n, &p) = " << min(&n,&p) << " \ n";
```

Donne comme résultat

```
&n = 0xbfa08998 &p = 0xbfa0899c  
min (&n, &p) = 0xbfa08998
```

Utilisations du patron de fonctions

Le code:

```
char * adr1 = "monsieur", * adr2 = "bonjour" ;  
cout << "min(adr1, adr2) = " << min(adr1, adr2);
```

Donne comme résultat

```
min (adr1, adr2) = monsieur
```

- L'appel `min(adr1,adr2)` génère la fonction

```
char * min (char * a, char * b)  
{  
    if (a < b) return a ;  
    else return b ;  
}
```
- La comparaison `a<b` porte sur les valeurs des pointeurs `a` et `b`.
- L'opérateur `<<` (affichage) porte sur les chaînes situées aux adresses `a` et `b`.

Utilisations du patron de fonctions

Application au type `class`

- Pour pouvoir appliquer le patron `min` à une classe, il faut que l'opérateur `<` puisse s'appliquer à deux opérandes de ce type classe.
 - Il faut le surcharger!!
- Considérons la classe `vect`

```
class vect {  
    int x, y ;  
    public :  
    vect (int abs=0, int ord=0) { x=abs ; y=ord; }  
    void affiche () { cout << x << " " << y; }  
    friend int operator < (vect, vect) ;  
} ;  
int operator < (vect a, vect b)  
{  
    return a.x*a.x + a.y*a.y < b.x*b.x + b.y*b.y ;  
}
```

Utilisations du patron de fonctions

Application au type **class**

Exemple d'utilisation :

```
main()
{
    vect u (3, 2), v (4, 1), w ;
    w = min (u, v) ;
    cout << " min(u, v) = ";
    w.affiche() ;
}
```

Sortie :

min (u, v) = 3 2

- Si nous cherchons à appliquer notre patron **min** à une classe pour laquelle l'opérateur **<** n'est pas défini, le compilateur le signalera.

Utilisation des paramètres de type dans la définition d'un patron

- Un patron de fonctions peut comporter un ou plusieurs paramètres de type.
- Chaque paramètre de type doit être précédé du mot-clé **class** :

```
template <class T, class U> fct (T a, T * b, U c)
{
...
}
```

- Les paramètres de type peuvent intervenir
 - Dans l'en-tête ;
template <class T, class U> fct (T a, T * b, U c)
 - Dans des déclarations de variables locales ;
T x ; // variable locale x de type T
 - Dans les instructions exécutables. // allocation tableau de 10 éléments de type T
adr = new T [10] ;

Utilisation des paramètres de type dans la définition d'un patron

Exemple d'utilisation de paramètres de type :

```
template <class T, class U> fct (T a, T * b, U c)
{
    T x ; // variable locale x de type T
    U * adr ; // variable locale adr de type U *
    ...
    adr = new T [10] ; // allocation tableau de
    // 10 elements de type T
    ...
    n = sizeof (T) ;
    ...
}
```

- Remarque: il est nécessaire que chaque paramètre de type apparaisse au moins une fois dans l'en-tête du patron.

Identification des paramètres de type d'une fonction patron

- Lors d'un appel d'un patron, il doit y avoir une correspondance absolue des types.

- Exemple :

```
template <class T> T min (T a, T b)
{ if (a < b) return a ;
  else return b ;
}
```

...

```
int n ; char c ; unsigned int q ;
const int ci1 = 10, ci2 = 12 ;
int t[10] ;
int * adi ;
```

...

```
min (n, c) // erreur
min (n, q) // erreur
min (n, ci1) // erreur : const int et int ne correspondent pas
min (ci1, ci2) // min (const int, const int)
min (t, adi) // min (int *, int *) car ici, t est converti
// en int *, avant appel
```

Identification des paramètres de type d'une fonction patron

- Il est possible d'intervenir sur ce mécanisme d'identification de type.
- Autoriser la non correspondance absolue entre les types!!
- Exemple :
- `min<int> (c, n) /* force l'utilisation de min<int>, et donc la conversion de c en int ; le résultat sera de type int */`
- `min<char> (q, n) /* force l'utilisation de min<char>, et donc la conversion de q et de n en char ; le résultat sera de type char */`

Patrons de classes: Introduction

- Comme les patrons de fonctions, C++ permet de définir des patrons de classes (appelés aussi les classes génériques).
- Ecriture d'une seule classe que le compilateur puisse adapter automatiquement a différents types.

Création d'un patron de classes

- Création d'une classe **point**

- Les coordonnées des points sont de type **int**.

```
class point {  
    int x ; int y ;  
    public :  
    point (int abs=0, int ord=0) ;  
    void affiche () ;  
    // .....  
}
```

- Les coordonnées des points sont de type **float**.

```
class point {  
    float x ; float y ;  
    public :  
    point (float abs=0, float ord=0) ;  
    void affiche () ;  
    // .....  
};
```

Création d'un patron de classes

- Définir une classe pour chaque type des coordonnées!!! Remplacer le mot **int** dans la classe **point** par le type voulu.
- Les choses peuvent être simplifiées en définissant un seul patron de classe :

```
template <class T> class point {  
    T x ; T y ;  
    public :  
    point (T abs=0, T ord=0) ;  
    void affiche () ;  
} ;
```
- Définition des fonctions membres de la classe, comment?
Il y a deux possibilités.
 - Fonction en ligne.
 - N'est pas fonction en ligne.

Définition des fonctions membres

- Une fonction en ligne: il suffit tout simplement d'utiliser le paramètre **T** a bon escient.

```
point (T abs=0, T ord=0)
```

```
{  
x = abs ; y = ord ;  
}
```

Le patron de classes correspondant:

```
template <class T> class point
```

```
{  
T x ; T y ;  
public :  
point (T abs=0, T ord=0)  
{  
x = abs ; y = ord ;  
}  
void affiche () ;  
} ;
```

Définition des fonctions membres

- fonction membre définie en dehors de la définition de classe:
Il faut rappeler au compilateur :
 - que dans la définition de cette fonction, vont apparaitre des paramètres de type. Il faut utiliser de nouveau `template <class T>`
 - le nom du patron concerné. Supposons que nous souhaitons définir la fonction `affiche`. Son nom sera `point<T>::affiche ()`
- si on définit la fonction `affiche` en dehors de la classe, son en-tête sera comme suit
`template <class T> void point<T>::affiche ()`

Définition des fonctions membres

Le patron **point** pourrait être défini comme suit:

```
#include < iostream >
using namespace std ;
// creation d'un patron de classe
template <class T> class point
{
    T x ; T y ;
public :
    // Definition en ligne
    point (T abs=0, T ord=0)
    {
        x = abs ; y = ord ;
    }
    void affiche () ;
};
// definition de la fonction affiche
// en dehors de la classe
template <class T> void point<T>::affiche ()
{
    cout << " Coordonnees : " << x
    << "" << y << " \n" ; }
}
```

Utilisation d'un patron de classes

- Pour une déclaration telle que
`point <int> ai ;`
le compilateur instancie la définition d'une classe `point` pour `T = int`.
- De même pour la déclaration
`point <double> ad ;`
instanciation de la définition la classe `point` pour `T = double`.
- Déclaration en spécifiant les arguments a fournir au constructeur:
`point <int> ai (3, 5) ;`
`point <double> ad (3.5, 2.3) ;`

Exemple d'un patron de classes

Création et utilisation

```
#include < iostream >
using namespace std ; // creation d'un patron
de classe
template <class T> class point{
T x ; T y ;
public :
point (T abs=0, T ord=0)
{ x = abs ; y = ord ; }
void affiche () } ;
template <class T>
void point<T>::affiche ()
{ cout << " Coordonnees : "
<< x << " " << y << " \n" ; }
main (){
point <int> ai (3, 5) ;
ai.affiche () ;
point <char> ac (d, y) ;
ac.affiche () ;
point <double> ad (3.5, 2.3) ;
ad.affiche () ;}
```

Sortie :

coordonnees : 3 5

coordonnees : d y

coordonnees : 3.5 2.3

Le patron de classes **point** est doté d'un constructeur en ligne et d'une fonction membre (**affiche**) non en ligne.

Les paramètres de type d'un patron

- Les paramètres de type d'un patron de classes peuvent être en nombre quelconque.
- Ils sont utilisés dans la définition du patron de classes.

Patron de classes avec 3 paramètres de type

```
template <class T, class U, class V>
```

```
class essai {
```

```
    T x ;
```

```
    U t[5] ;
```

```
    ...
```

```
    V fm1 (int, U) ;
```

```
};
```

- Trois paramètres de type **T**, **U**, **V**
`template <class T, class U, class V>`
- Un membre **x** de type **T**
- Un tableau **t** de 5 éléments de type **U**
- Déclaration d'une fonction membre recevant 2 arguments de type **int** et **U** et renvoyant un résultat de type **V**

Instanciation d'une classe patron

- Rappel : Une classe patron est une instance particulière d'un patron de classes.
- Une classe patron se déclare en fournissant a la suite du nom de patron un nombre d'arguments effectifs (noms de types) égal au nombre de paramètres figurant dans la liste du patron.

essai <int, float, int> ce1 ;

essai <int, int *, double > ce2 ;

essai <char *, int, obj> ce3 ;

- Dans la dernière déclaration (instanciation de la classe patron), on suppose que **obj** est un type préalablement défini.
- Possible d'utiliser comme paramètre de type effectif un type instancie a l'aide d'un patron de classes.

essai <float, int<int>, double> ce4 ;

essai <point<int>, point<float>, char *> ce5;

L'héritage

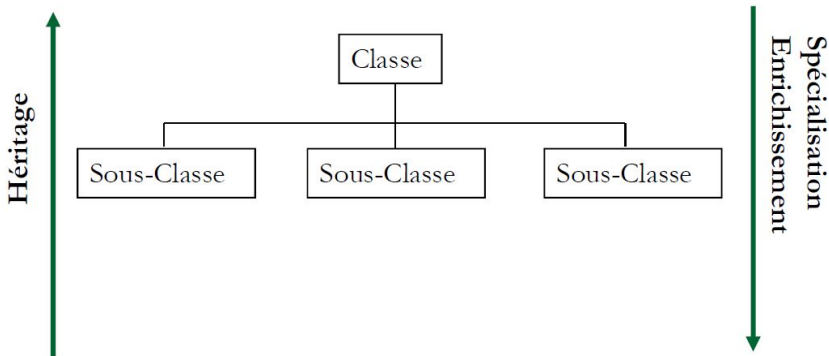
Objectifs du chapitre

Ce chapitre a pour but :

- de présenter en détail la notion fondamentale d'héritage
- de vous présenter encore quelques subtilités au sujet des constructeurs et des destructeurs

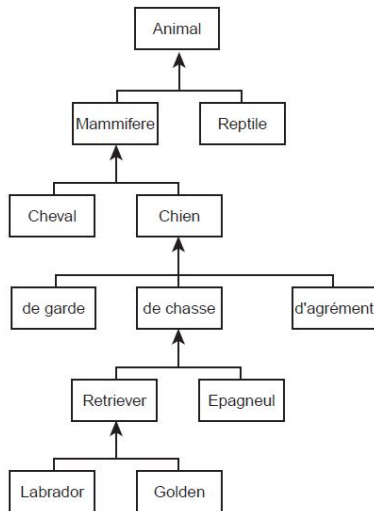
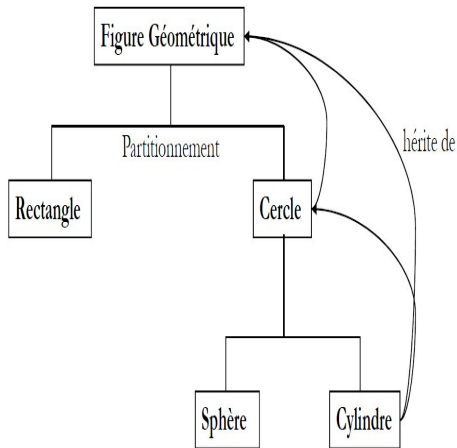
Héritage

- Après les notions d'encapsulation et d'abstraction, le troisième aspect essentiel des objets est la notion d'héritage
- L'héritage est une technique extrêmement efficace pour créer des classes plus spécialisées, appelées sous-classes, à partir de classes plus générales déjà existantes, appelées super-classes.
- Elle représente la relation $\ll \text{est} - \text{un} \gg$



Hiérarchie de classes

un exemple



- plus précisément, lorsqu'une sous-classe C1 est créée à partir d'une classe C, C1 va hériter de (i.e. recevoir) l'ensemble :
 - des attributs de C
 - des méthodes de C (sauf les constructeurs/destructeurs)
- les attributs et méthodes de C vont être disponibles pour C1 sans que l'on ait besoin de les redéfinir explicitement dans C1.
- De plus :
 - des attributs et/ou méthodes supplémentaires peuvent être définis par la sous-classe C1
 - ces membres constituent l'enrichissement apporté par cette sous-classe.

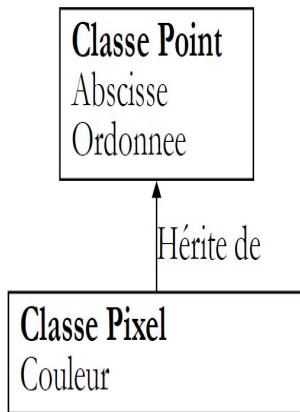
L'héritage permet donc :

- d'expliciter des relations structurelles et sémantiques entre classes
- de réduire les redondances de description et de stockage des propriétés

Héritage

Supposons par exemple que l'on veuille étendre la classe **Point** précédemment définie en lui ajoutant le nouvel attribut **couleur**.

Une façon de procéder est de créer une nouvelle classe, par exemple **Pixel**, définie comme une sous-classe de **Point** et contenant le nouvel attribut **couleur**.
on évite ainsi de dupliquer inutilement du code commun aux classes **Point** et **Pixel**.



- par transitivité, les instances d'une sous-classe possèdent : les attributs et méthodes de l'ensemble des classes parentes (classe parente, classe parente de la parente, etc . . .)
- la notion enrichissement par héritage :
 - crée un réseau de dépendances entre classes,
 - ce réseau est organisé en une structure arborescente où chacun des nœuds hérite des propriétés de l'ensemble des nœuds du chemin remontant jusqu'à la racine.
- ce réseau de dépendance définit une hiérarchie de classes

Définition d'une sous-classe en C++ :

Syntaxe :

```
class NomClasseEnfant : public NomClasseParente
{
/* Déclaration des attributs et méthodes
spécifiques à la sous-classe */
//...
};
```

Exemple :

```
class Pixel : public Point
{
Couleur couleur;
//...
};
```

Accès aux membres d'une sous-classe

Jusqu'à maintenant, l'accès aux membres (attributs et méthodes) d'une classe pouvait être :

- soit public : visibilité totale à l'intérieur et à l'extérieur de la classe (mot-clé **public**).
- soit privé : visibilité uniquement à l'intérieur de la classe (mot-clé **private** ou par défaut).

Un troisième type d'accès régit l'accès aux attributs/méthodes au sein d'une hiérarchie de classes :

- l'accès protégé : assure la visibilité des membres d'une classe dans les classes de sa descendance. Le mot clé est **<< protected >>**.

Définition des niveaux d'accès

L'ordre de définition conseillé est le suivant :

```
class NomClasse[: public NomClasseParente]
{
// par défaut : privé
public:
// attributs et méthodes publics
protected:
// attributs et méthodes protégés
private:
// attributs et méthodes privés
};
```

Mais il peut y avoir plusieurs zones publiques, protégées ou privées dans une même définition de classe.

Accès protégé (1)

Le niveau d'accès protégé correspond à une extension du niveau privé aux membre des sous-classes

Exemple :

```
class Rectangle
{
public:
Rectangle(): largeur(1.0), hauteur(2.0) {}
protected:
double largeur; double hauteur;
};
class RectangleColore : public Rectangle
{
public:
...
void carre() { largeur = hauteur; }
protected:
Couleur couleur;
};
```

Accès protégé (2)

Le niveau d'accès protégé correspond à une extension du niveau privé aux membre des sous-classes... mais uniquement dans ce rôle (de sous-classe) pas dans le rôle de classe extérieure :

Exemple :

```
class A {  
...  
protected: int a;  
private: int prive;  
};  
class B: public A {  
public:  
...  
void f(B autreB, A autreA, int x) {  
a = x; // OK A::a est protected => accès possible  
// prive = x; // erreur : A::prive est private  
// a += autreB.prive; // erreur (même raison)  
a += autreB.a; // OK : dans la même classe (B)  
}  
};
```

Restriction des accès lors de l'héritage

Les niveaux d'accès peuvent être modifiés lors de l'héritage

Syntaxe :

```
class ClasseEnfant: [acces] classeParente
{
/* Déclaration des membres
spécifiques à la sous-classe */
//...
};
```

où accès est le mot-clé **public**, **protected** ou **private**. Par défaut, l'accès est privé.

Restriction des accès lors de l'héritage(2)

Récapitulatif des changements de niveaux d'accès aux membres hérités, en fonction du niveau initial et du type d'héritage :

		Accès initial		
		public	protected	private
Héritage	public	public	protected	Pas d'accès
	protected	protected	protected	Pas d'accès
	private	private	private	Pas d'accès

Le type d'héritage constitue une limite supérieure à la visibilité.

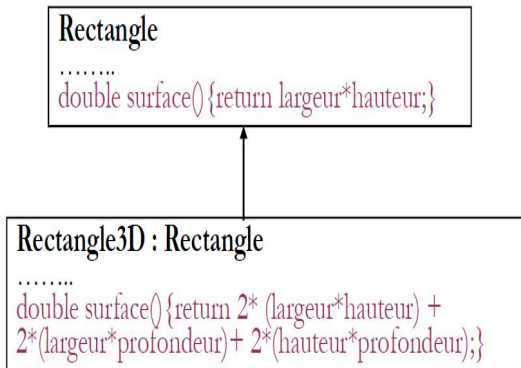
- Membres publics : accessibles pour les utilisateurs de la classe
- Membres protégés : accessibles aux autres programmeurs pour d'éventuelles extensions par héritage de la classe
- Membres privés : utilisable par les fonctions membres et les classes et opérations amies de cette seule classe, à l'exception des sous-classes;

Masquage dans une hiérarchie

- Masquage : un identificateur qui en cache un autre
- Situation possible dans une hiérarchie :
 - Même nom d'attribut/méthode utilisé sur plusieurs niveaux
 - Peu courant pour les attributs
 - Très courant et pratique pour les méthodes
- Exemple :
 - **Rectangle3D** hérite de **Rectangle**
 - calcul de la surface pour les **Rectangle3D**
 $2 * (\text{largeur} * \text{hauteur}) + 2 * (\text{largeur} * \text{profondeur}) + 2 * (\text{hauteur} * \text{profondeur})$
 - calcul de la surface pour tous les autres **Rectangle** :
 $(\text{largeur} * \text{hauteur})$
- Faut-il re-concevoir toute la hiérarchie ?
 - Non, on ajoute simplement une méthode surface spéciale à **Rectangle3D**

Masquage dans une hiérarchie (2)

- La méthode surface de **Rectangle3D** cache celle de **Rectangle**
- Un objet de type **Rectangle3D** n'utilisera donc jamais la méthode surface de la classe **Rectangle**
- Vocabulaire OO :
 - Méthode héritée = méthode générale, méthode par défaut
 - Méthode qui cache la méthode héritée = méthode spécialisée



Masquage dans une hiérarchie (3)

Code C++ de l'exemple :

```
class Rectangle{
public:
// les constructeurs seraient ici...
double surface() {return (largeur*hauteur);}
protected:
double largeur; double hauteur;
// le reste de la classe...
};
class Rectangle3D : public Rectangle{
public:
// les constructeurs seraient ici...
double surface(){
return(2.0*(largeur*hauteur) + 2.0*(largeur*profondeur)
+ 2.0*(hauteur*profondeur));
}
protected:
double profondeur;
// le reste de la classe...
};
```

Accès à une méthode masquée

- Il est parfois souhaitable d'accéder à une méthode/un attribut caché(e)
Exemple :
- Surface des **Rectangle3D** ayant une profondeur nulle (**largeur*hauteur**)
surface identique au calcul de surface pour les **Rectangle**
- Code désiré :
 - 1 Objet non-Rectangle3D :
Méthode générale (surface de Rectangle)
 - 2 Objet Rectangle3D :
Méthode spécialisée (surface de Rectangle3D)
 - 3 Objet Rectangle3D de profondeur nulle :
D'abord la méthode spécialisée
Ensuite appel à la méthode générale depuis la méthode spécialisée

Accès à une méthode masquée (2)

- Pour accéder aux attributs/méthodes caché(e)s de la super-classe : on utilise l'opérateur de résolution de portée
- Syntaxe : **NomClasse::methode** ou attribut
- Exemple : **Rectangle::surface()**

```
class Rectangle3D : public Rectangle{  
    //... constructeurs, attributs comme avant  
    double surface () {  
        if (profondeur == 0.0)  
            return Rectangle::surface();  
        else  
            return(2.0*(largeur*hauteur) + 2.0*(largeur*profondeur)  
                + 2.0*(hauteur*profondeur));  
    }  
};
```

Constructeurs et destructeurs

La construction d'un objet d'une classe dérivée comporte:

- la réservation de la place pour l'objet;
- l'appel d'un constructeur approprié (constructeur par défaut) de la classe de base;
- appel des constructeurs pour les membres objets;
- l'exécution du corps de la fonction constructeur.

Un constructeur autre que celui par défaut est indiqué dans la liste d'initialisation.
La destruction d'un objet d'une classe dérivée:

- appel du destructeur de la classe dérivée;
- appel du destructeur de la classe de base.

Lors de l'instanciation d'une sous-classe, il faut initialiser :

- les attributs propres à la sous-classe
- les attributs hérités des super-classes

MAIS...

...il ne doit pas être à la charge du concepteur des sous-classes de réaliser lui-même l'initialisation des attributs hérités

L'accès à ces attributs peut notamment être interdit !

L'initialisation des attributs hérités doit donc se faire au niveau des classes où ils sont explicitement définis.

Solution : l'initialisation des attributs hérités doit se faire en invoquant les constructeurs des super-classes.

Constructeurs et héritage (2)

- L'invocation du constructeur de la super-classe se fait au début de la section d'appel au constructeurs des attributs. Syntaxe :

SousClasse(liste de paramètres):

SuperClasse(Arguments),attribut1(valeur1)...attributN(valeurN)

{

// corps du constructeur

}

- Lorsque la super-classe admet un constructeur par défaut, l'invocation explicite de ce constructeur dans la sous-classe n'est pas obligatoire
- Le compilateur se charge de réaliser l'invocation du constructeur

Constructeurs et héritage (3)

Exemple :

```
class point{
int x,y;
public : point(int abs=0, int ord =0){
x=abs; y=ord;
}
~point(){}
};
class pixel : public point {
short couleur;
public :
pixel(int abs=0,int ord=0,short cl=1):point(abs,ord),
couleur(cl){
}
~pixel(){
}
};
```

Constructeurs et héritage (4)

- Si la classe parente n'admet pas de constructeur par défaut, l'invocation explicite d'un de ses constructeurs est obligatoire dans les constructeurs de la sous-classe
- La sur-classe doit admettre au moins un constructeur explicite.

Exemple :

```
class Rectangle {  
protected: double largeur; double hauteur;  
public:  
Rectangle(double l, double h) : largeur(l), hauteur(h)  
{  
// le reste de la classe...  
};  
class Rectangle3D : public Rectangle {  
protected: double profondeur;  
public:  
Rectangle3D(double l, double h, double p)  
: Rectangle(l,h), profondeur(p){}  
// le reste de la classe...  
};
```

Constructeurs et héritage (5)

Autre exemple (qui ne fait pas la même chose) :

```
class Rectangle{  
protected: double largeur; double hauteur;  
public:  
Rectangle() : largeur(0.0), hauteur(0.0)  
{  
// le reste de la classe...  
};  
class Rectangle3D : public Rectangle {  
protected: double profondeur;  
public:  
Rectangle3D(double p)  
: profondeur(p)  
{  
// le reste de la classe...  
};
```

Ici il n'est pas nécessaire d'invoquer explicitement le constructeur de la classe parente puisque celle-ci admet un constructeur par défaut.

Héritage multiple

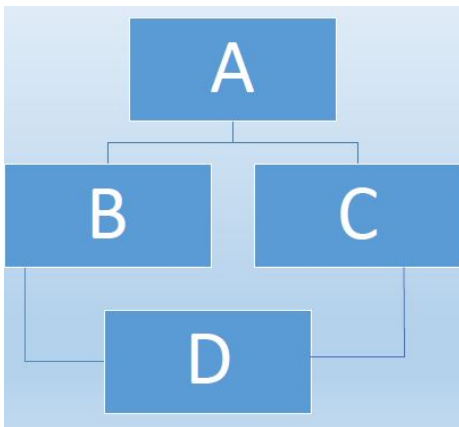
Une classe peut hériter de plusieurs classes. Les classes de bases sont alors énumérées dans la définition.

class D : public B1, public B2 {...}

Exemple:

```
class point{
int x,y;
public:
point(...){...}
~point(){...}
void afficher(){...}
};
class coul {
short couleur;
public :
coul(...){...}
~coul(...){...}
void afficher(){...}
};
// classe dérivée
class pixel : public point, public coul {...
pixel(...):point(...),coul(...){
...}
void afficher(){...} };
```

Héritage multiple : Héritage en Diamant



La difficulté de ce type d'héritage commence quand deux classes parentes héritent d'une même classe. On se retrouve alors dans la situation suivante :

- C hérite de A
- B hérite de A
- D hérite de B et de C

B et C vont donc construire chacun une classe A. Il y a donc un risque de conflit quand, dans D, on veut accéder à une donnée de A.

Solution : Les Classes Virtuelles!!

Le polymorphisme

Qu'est ce que le polymorphisme

L'édition de lien est l'opération qui associe, à l'appel d'une fonction, l'adresse du code de la fonction.

On distingue

- La liaison statique (ou précoce): effectuée à la compilation
- La liaison dynamique (ou tardive) : effectuée à l'exécution.

Qu'est ce que le polymorphisme (2)

Le polymorphisme est la capacité, pour une fonction, d'être interprétée différemment en fonction de l'objet auquel elle s'applique.

On distingue:

- Le polymorphisme statique : c'est la surcharge. Le sens de la fonction est déterminé statiquement par la signature des arguments à l'appel.
- Le polymorphisme dynamique : c'est le véritable polymorphisme. Le sens de la fonction est déterminé par le type effectif de l'objet à l'appel.

Qu'est ce que le polymorphisme (3)

Le polymorphisme ne s'applique qu'aux méthodes d'une classe.

Avantages:

- encourage l'abstraction,
- facilite l'extensibilité.

Héritage (rappel)

Dans une hiérarchie de classes, la sous-classe hérite de la super-classe :

- tous les attributs/méthodes (sauf constructeurs et destructeur)
- le type

L'héritage est transitif.

Héritage du type : on peut affecter un objet de type sousclasse à une variable de type super-classe.

Exemple:

```
Shape fig1;
```

```
Rectangle fig2;
```

```
fig1 = fig2;
```

Exemple

```
class Base
{
public: void show(){cout << " Base \n";}
};
class Derv1 : public Base // derived class 1
{
public: void show(){cout << " Derv1 \n";}
};
class Derv2 : public Base // derived class 2
{
public: void show(){cout << " Derv2 \n";}
};
int main()
{
Derv1 dv1; // object of derived class 1
Derv2 dv2; // object of derived class 2
Base * ptr; // pointer to base class
ptr = &dv1; //put address of dv1 in pointer
ptr->show(); // execute show
ptr = &dv2;
ptr->show();
return 0;
}
```

Résultat d'exécution:
Les instances de Derv1 et
Derv2 sont substituables
aux instances de Base :
Compatibilité
ascendante des types

- En POO, le polymorphisme est le fait que : les instances d'une sous-classe soient substituables aux instances des classes de leur ascendance, en argument d'une méthode ou lors d'affectations
- La mise en œuvre se fait par :
 - les mécanismes de l'héritage dans les hiérarchies de classes,
 - la résolution dynamique des liens : le choix des méthodes à invoquer se fait lors de l'exécution du programme en fonction de la nature réelle des instances concernées

Résolution des liens

- Une instance de sous-classe **B** est substituable à une instance de super-classe **A**.
- Que se passe-t-il lorsque **B** redéfinit une méthode de **A** ?

```
class A{  
public:  
void affiche(){cout << " A" << endl;}  
};  
class B: public A{  
public:  
void affiche(){cout << " B" << endl;}  
};  
int main(){  
A aa;  
B bb;  
aa = bb;  
aa.affiche();  
}
```

Résolution des liens (2)

- En C++, c'est le type de la variable qui détermine la méthode à exécuter .
- Résolution statique des liens.
- Ligature statique (early binding)

```
class A{  
public:  
void affiche(){cout << " A" << endl;}  
class B: public A{  
public:  
void affiche(){cout << " B" << endl;}  
int main(){  
A aa;  
B bb;  
aa = bb;  
aa.affiche();  
}
```


Résolution dynamique des liens

- Il pourrait dans certains cas sembler plus naturel de choisir la méthode correspondante à la nature réelle de l'instance.
`aa.affiche(); -> bb.affiche();`
- Dans ces cas, il faut permettre la résolution dynamique des liens : Le choix de la méthode à exécuter se fait à l'exécution, en fonction de la nature réelles des instances.

Méthodes virtuelles

- En C++, on indique au compilateur qu'une méthode peut faire l'objet d'une résolution dynamique des liens en la déclarant comme virtuelle (mot clé **virtual**).
- cette déclaration doit se faire dans la classe la plus générale qui admet cette méthode (c'est-à-dire lors du prototypage d'origine)
- les redéfinitions éventuelles dans les sous-classes seront aussi considérées comme virtuelles par transitivité.

Syntaxe :

virtual Type nom_fonction(liste d'arguments) [const];

Exemple :

```
class A {  
    virtual void afficher() const {  
        cout << " A" << endl; }  
};
```

Méthodes virtuelles (2)

```
class Base{
public:
virtual void affiche(){
    cout << " Base \ n";
}};
class Derv : public Base{
public:
void affiche(){
    cout << " Derv \ n";
}};
void F(Base t){
    t.affiche();}
main(){
    Base b;
    Derv d;
    F(b);
    F(d);
}
```

Méthodes virtuelles (3)

- Ici la méthode **F** prend un argument par valeur. Elle va donc s'exécuter sur une copie de cet argument.
- Il faut passer **t** par référence pour que la méthode **F** agisse sur l'instance d'origine !

```
void F(Base & t){  
    t.affiche();  
}  
main(){  
    Base b;  
    Derv d;  
    F(b);  
    F(d);  
    return 0;  
}
```

Méthodes virtuelles (4)

- En résumé : Lorsqu'une méthode virtuelle est invoquée à partir d'une référence ou d'un pointeur vers une instance, c'est la méthode associée au type réel de l'instance qui sera exécutée.
- Attention !
 - Un constructeur ne peut pas être virtuel
 - on ne peut pas invoquer de méthode virtuelle dans un constructeur (plus exactement : l'aspect polymorphique est ignoré dans le constructeur, seule l'instance correspondant à la classe courante est appelée)
 - il est conseillé de toujours définir les destructeurs comme virtuels

- Au sommet d'une hiérarchie de classe, il n'est pas toujours possible de :
 - donner une définition générale de certaines méthodes, compatibles avec toutes les sous-classes,
 - ...même si l'on sait que toutes ces sous-classes vont effectivement implémenter ces méthodes

Méthodes virtuelles pures (2)

Une méthode virtuelle pure, ou abstraite, est incomplètement spécifiée :

- elle n'a pas de définition dans la classe où elle est introduite (pas de corps)
- elle sert à signaler aux sous-classes qu'elles doivent redéfinir la méthode virtuelle héritée

Syntaxe :

```
virtual Type nom_methode(liste d'arguments) = 0;
```

Méthodes virtuelles pures (3)

```
class Base{  
public:  
virtual void affiche()=0;  
class Derv : public Base{  
public:  
void affiche(){  
    cout << " Derv \n";  
}  
};
```


- Une classe abstraite est une classe contenant au moins une méthode virtuelle pure.
- Elle ne peut être instanciée
- Ses sous-classes restent abstraites tant qu'elles ne fournissent pas les définitions de toutes les méthodes virtuelles pures dont elles héritent

Généralités sur STL

Le C++, tout comme le C et de nombreux langages, possède une librairie disponible dès l'installation connue sous le nom de librairie standard. On connaît déjà quelques éléments de cette librairie notamment par les include `<cmath>` `<iostream>` `<string>` etc.

Une partie de cette librairie concerne des versions stables, optimisées, et testées de conteneurs, d'itérateurs et d'algorithmes sur ces conteneurs. Cette librairie est connue sous le nom de **Standard Template Library** ou **STL**. C'est une librairie puissante conçue à base de classe **template** dont nous allons maintenant étudier quelques éléments.

La STL: La classe Vector

```
#include<iostream>
#include<vector>
int main()
{
    double tab[5] = {1, 2, 3, 4, 5};
    std::vector<double> v(tab, tab + 5);
    v.push_back(10);
    std :: cout << v[0] << std :: endl;
    std :: cout << v.back() << std :: endl;
    v.pop_back();
    std :: cout << v.back() << std :: endl;
}
```

Nous avons déjà plus ou moins construit une classe **vector** telle qu'implémentée dans la **STL**. On en trouve la documentation complète sur cplusplus.com [http :://www cplusplus com/reference/vector/vector/](http://www.cplusplus.com/reference/vector/vector/).
Ci contre, un exemple d'utilisation

La STL: La classe List

```
#include<iostream>
#include<list>
int main()
{
double tab[5] = {1, 2, 3, 4, 5};
std::list<double> v(tab, tab + 5);
v.push_back(10);
/* v[0] = 5; N'EXISTE PAS POUR UNE
LISTE */
std :: cout << v.front() << std :: endl;
std :: cout << v.back() << std :: endl;
v.pop_back();
std :: cout << v.back() << std :: endl;
}
```

La classe `std::list` s'apparente en terme d'utilisation à la classe `vector`. Il y a néanmoins des différences d'implémentation entre un `vector` et une `list` qui font qu'on préférera l'une ou l'autre classe en fonction de l'utilisation de celle ci.

La STL: La classe List

```
#include<iostream>
#include<list>
int main()
{
double tab[5] = {1, 2, 3, 4, 5};
std::list<double> v(tab, tab + 5);
v.push_back(10);
/* v[0] = 5; N'EXISTE PAS POUR UNE
LISTE */
std :: cout << v.front() << std :: endl;
std :: cout << v.back() << std :: endl;
v.pop_back();
std :: cout << v.back() << std :: endl;
}
```

La classe **list** telle qu'implémentée dans la **STL** est ce qu'on appelle une liste doublement chaînée. On remarque qu'il n'y a pas de surcharge de l'opérateur []. En effet, une liste a des avantages par rapports aux vecteurs/tableaux/espaces mémoire contiguë si cet ensemble d'éléments est appelé à grandir/diminuer ou si on veut insérer un élément dans le tableau.

La STL: La classe List

```
#include<iostream>
#include<list>
int main()
{
double tab[5] = {1, 2, 3, 4, 5};
std::list<double> v(tab, tab + 5);
v.push_back(10);
/* v[0] = 5; N'EXISTE PAS POUR UNE
LISTE */
std :: cout << v.front() << std :: endl;
std :: cout << v.back() << std :: endl;
v.pop_back();
std :: cout << v.back() << std :: endl;
}
```

Par contre, pour accéder au n^{ème} élément d'une liste, il faut la parcourir du début, élément par élément. Le temps d'accès est donc linéaire par rapport à la taille de la liste. En notation de Landau $O(n)$. Dans un tableau, le temps d'accès est constant $O(1)$. Pour agrandir une liste, par contre, le temps est constant, alors qu'il est difficile d'augmenter le nombre d'éléments d'un tableau. On doit copier les éléments de l'ancien tableau dans le nouveau avant de libérer l'ancien espace mémoire etc.

La STL: La classe Map

```
#include<iostream>
#include<map>
#include<string>
int main()
{
    std::map<std::string, double> m;
    m["Charles"] = 5.0;
    m["Toto"] = 10;
    m["Sally"] = 15;
    std :: cout << m.size() << std :: endl;
    std :: cout << m["Sally"] << std ::
    endl;
}
```

La classe `std::map` est un autre conteneur qui permet d'associer une clé à une valeur. On parle de tableaux associatifs ou de dictionnaires. Comme c'est une classe `template` le type des clés/valeurs peut être attribué lors de l'écriture du code. L'opérateur `[]` est cette fois ci surchargé pour accéder à la valeur d'une clé.

Il existe d'autres conteneurs dans la **STL** et nous ne les verrons pas tous, on pourra néanmoins se reporter à la référence sur cplusplus.com pour connaître les autres types en fonction des besoins.

Pour homogénéiser les actions possibles sur les différents conteneurs, il est apparu la notion d'itérateurs. Un itérateur qui peut être vu comme une généralisation de la notion de pointeur, permet de parcourir un conteneur. Il possède ces propriétés

- À chaque instant, un itérateur possède une valeur qui désigne un élément donné du conteneur. On dira qu'il `<< pointe >>` vers cet élément
- Un itérateur peut être incrémenté par l'opérateur `++`
- Il peut être déréférencé, c'est à dire que l'utilisation de l'opérateur `*` permet (comme sur un pointeur) d'accéder à la valeur courante de l'itérateur
- Deux itérateurs sur un même conteneur peuvent être comparés

La STL: Itérateur sur Vector

```
#include<vector>
#include<iostream>
int main()
{
    std::vector<double> v(5);
    std::vector<double>::iterator iv;
    double i = 0;
    for (iv = v.begin() ;
    iv != v.end () ;
    ++iv)
    {
        *iv = i++;
    }
    for (iv = v.begin();
    iv != v.end () ;
    ++iv)
    {
        std :: cout << *iv << " ; " ; }
    }
```

Un itérateur se déclare de la façon suivante

```
std::vector<double>::iterator iv;
```

La première partie est un **vector** de **double**.

Que signifie la deuxième partie ?

Les objets itérateurs sont définis à l'intérieur du conteneur qu'ils itèrent. On dit qu'il s'agit de classes imbriquées, ou << nested class >>. En effet, on peut déclarer une classe à l'intérieur d'une autre classe. Pour s'adresser à cette classe imbriquée, on va faire appel à l'opérateur <<::>>

La STL: Itérateur sur Vector

```
#include<vector>
#include<iostream>
int main()
{
    std::vector<double> v(5);
    std::vector<double>::iterator iv;
    double i = 0;
    for (iv = v.begin() ;
        iv != v.end () ;
        ++iv)
    {
        *iv = i++;
    }
    for (iv = v.begin();
        iv != v.end () ;
        ++iv)
    {
        std :: cout << *iv << " ; " ; }
    }
```

Tous les conteneurs fournissent deux valeurs particulières, sous forme d'itérateur permettant de fournir un début et une fin. Ainsi, on initialise l'itérateur avec comme valeur l'itérateur pointant sur le début du vecteur. Comme on peut comparer l'égalité ou l'inégalité de deux itérateurs on arrête la boucle **for** quand l'itérateur est égal à l'itérateur pointant sur la fin du vecteur. Attention **end()** est un itérateur particulier qui ne pointe pas sur le dernier élément, mais un cran plus loin. De sorte que pour un conteneur vide, **begin()** et **end()** sont égaux.

La STL: Itérateur sur Vector

```
#include<vector>
#include<iostream>
int main()
{
    std::vector<double> v(5);
    std::vector<double>::iterator iv;
    double i = 0;
    for (iv = v.begin() ;
        iv != v.end () ;
        ++iv)
    {
        *iv = i++;
    }
    for (iv = v.begin();
        iv != v.end () ;
        ++iv)
    {
        std :: cout << *iv << " ; " ; }
    }
```

Pour passer à l'élément suivant, l'itérateur peut être incrémenté grâce à l'opérateur `<< ++ >>`. Attention cela ne veut pas dire qu'il existe un opérateur `<< -- >>`. On remarquera aussi que l'on écrit `++iv` et non `iv++`. Pour comprendre pourquoi, on se souviendra de la surcharge de l'opérateur d'incrément et que la notation postfixée entraîne plus d'opérations que la notation préfixée

La STL: Itérateur sur Vector

```
#include<vector>
#include<iostream>
int main()
{
    std::vector<double> v(5);
    std::vector<double>::iterator iv;
    double i = 0;
    for (iv = v.begin();
    iv != v.end ();
    ++iv)
    {
        *iv = i++;
    }
    iv = v.begin(); for (int i =
    0;i<v.size();i++)
    {
        std :: cout << *(iv + i) << " "; }
    }
```

Pour accéder à la valeur pointée par l'itérateur on le dérèfère grâce à l'opérateur `<< * >>`. On peut ainsi modifier ou accéder à cette valeur. Pour les vecteurs, il existe aussi une surcharge de l'opérateur `<< [] >>` permettant d'accéder immédiatement à un élément donné. Cela entraîne la possibilité d'une arithmétique des itérateurs sur vecteur. Ainsi, dans l'exemple ci contre, `*(iv+i)` est équivalent à `v[i]`

La STL: Itérateur sur List

```
#include<list>
#include<iostream>
int main()
{
    std::list<int> lis;
    lis.push_back(2);
    lis.push_back(3);
    std::list<int>::iterator il;
    for (il=lis.begin(); il != lis.end(); ++il);
    std :: cout << *il << std :: endl;
    // NE COMPILE PAS
    //std :: cout << *(lis.begin() + 1) <<
    std :: endl;
}
```

Les itérateurs sur **list** se déclarent de la même façon que les itérateurs sur **vector**. De même, une liste possède deux fonctions particulières **begin()** et **end()** qui permettent d'initialiser et de comparer un itérateur sur **list**. De même, l'opérateur de déréférencement permet d'accéder à la valeur pointée par l'itérateur

La STL: Itérateur sur List

```
#include<list>
#include<iostream>
int main()
{
    std::list<int> lis;
    lis.push_back(2);
    lis.push_back(3);
    std::list<int>::iterator il;
    for (il=lis.begin(); il != lis.end(); ++il)
        std :: cout << *il << std :: endl;
    // NE COMPILE PAS
    //std :: cout << *(lis.begin() + 1) <<
    std :: endl;
}
```

Par contre, il n'y a pas d'opérateur `[]` sur une liste. Une liste ne s'utilise pas comme un **vector**. Accéder directement à un élément n'est pas souhaitable car c'est une procédure lente. Il n'y a donc pas d'arithmétique sur les itérateurs de **list** on garde une cohérence entre le conteneur et son itérateur

La STL: Itérateur sur Map

```
#include<map>
#include<string>
#include<iostream>
int main()
{
    std::map<std::string, double> m;
    m["Charles"] = 5;
    m["Zoé"] = 6;
    m["Toto"] = 7;
    for (std::map<std::string,
double>::iterator im = m.begin();
im != m.end (); ++im)
    {
        std :: cout << (*im).first << " " <<
(*im).second << std :: endl;
    }
}
```

On définit ici une **map** templétée par une chaîne de caractères et un **double**. Dans la boucle **for**, on déclare et on initialise l'itérateur directement dans la boucle, comme on pourrait le faire avec un entier par exemple

```
$ ./a.exe
Charles 5
Toto 7
Zoé 6
```

La STL: Itérateur sur Map

```
#include<map>
#include<string>
#include<iostream>
int main()
{
    std::map<std::string, double> m;
    m["Charles"] = 5;
    m["Zoé"] = 6;
    m["Toto"] = 7;
    for (std::map<std::string,
double>::iterator im = m.begin();
im != m.end (); ++im)
    {
        std :: cout << (*im).first << " " <<
(*im).second << std :: endl;
    }
}
```

Ensuite, l' itérateur s'utilise de la même façon que ceux sur les **vector** ou les **list**. On peut également déréférencer l'itérateur sur **map**. Par contre, la valeur accédée est une paire

```
$ ./a.exe
Charles 5
Toto 7
Zoé 6
```

La STL: Itérateur sur Map

```
#include<map>
#include<string>
#include<iostream>
int main()
{
    std::map<std::string, double> m;
    m["Charles"] = 5;
    m["Zoé"] = 6;
    m["Toto"] = 7;
    for (std::map<std::string,
double>::iterator im = m.begin();
im != m.end (); ++im)
    {
        std :: cout << (*im).first << " " <<
(*im).second << std :: endl;
    }
}
```

Une paire est une classe qui existe également dans la **STL**. Il s'agit d'associer une clé à une valeur, et ce en utilisant les **template**. Ainsi la clé et la valeur peuvent être de type quelconque. Ainsi, le champ **first** d'une paire permet d'accéder à sa clé et le champ **second** permet d'accéder à la valeur

```
$ ./a.exe
Charles 5
Toto 7
Zoé 6
```

La STL: Itérateur sur Map

```
#include<map>
#include<string>
#include<iostream>
int main()
{
    std::map<std::string, double> m;
    m["Charles"] = 5;
    m["Zoé"] = 6;
    m["Toto"] = 7;
    for (std::map<std::string,
double>::iterator im = m.begin();
im != m.end (); ++im)
    {
        std :: cout << (*im).first << " " <<
        (*im).second << std :: endl;
    }
}
```

On remarque que l'affichage ne respecte pas l'ordre dans lequel on a rempli la **map**. Cela vient du fait qu'une **map** comme tous les conteneurs, est ordonnée. Ceci afin d'accélérer les algorithmes de recherche etc. Sur les chaînes de caractères, la relation d'ordre est triviale, il s'agit de l'ordre alphabétique.

```
$ ./a.exe
Charles 5
Toto 7
Zoé 6
```

La STL: For_each

```
#include <vector>
#include <algorithm>
#include <iostream>
void affiche( int i){
    std :: cout << i << " ";
}
int main()
{
    int myints [] =
    {23,45,56,77,63,87,54,74};
    std::vector<int> v(myints , myints+8)
    std::for_each(v.begin(), v.end()-2,
    affiche);
}
```

La **STL** contient également des algorithmes. Ces fonctions **template** travaillent sur des conteneurs et des itérateurs. On présente ici la fonction **for_each** qui exécute une fonction (ou un foncteur) passée en paramètre, entre deux bornes d'un conteneurs - c'est à dire entre deux valeurs d'itérateurs

```
$ ./a.exe
23;45;56;77;63;87;
```

La STL: Sort

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;
bool petit( int i, int j){
return i<j;
}
int main()
{
int myints [] =
{23,56,45,77,80,63,87,54,74};
vector<int> v(myints , myints+9);
sort(v.begin (), v.end (),petit);
for ( int i = 0; i < v.size(); ++i)
cout << v[i] << " - ";
cout << endl;
}
```

La fonction **sort** permet d'appliquer une fonction de tri sur un conteneur entre deux valeurs d'itérateurs. Le troisième argument est une fonction renvoyant vrai si deux éléments du tableau sont dans la bonne position

\$./a.exe

23-45-54-56-63-74-77-80-87-

La STL: Count/Count_if

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;
bool pair( int i){
return i%2==0;
}
int main()
{
int myints [] =
{23,56,45,77,80,63,87,54,23};
vector<int> v(myints , myints+9);
int c = count_if(v.begin (), v.end
()),pair);
cout << " Nbrespair : " << c << endl;
cout << "23 : " <<
count(v.begin(), v.end(), 23) << endl; }
```

La fonction count permet de compter le nombres d'occurrence d'un élément entre deux itérateurs d'un conteneur

La fonction count_if permet de compter le nombre d'éléments satisfaisant une condition. Cette condition prend la forme d'une fonction (ou d'un foncteur) renvoyant un bool et prenant en paramètre un élément du conteneur passée en paramètre

```
$ ./a.exe
```

```
Nbres   pair : 3
```

```
23 : 2
```

Gestion des exceptions

Les Exceptions

Pour gérer les situations inattendues lors de l'exécution d'un programme, il n'y a pas de solution simple. On peut, lors de chaque instruction qui peut provoquer une erreur d'exécution, faire des vérifications systématiques. Cela reviendrait à de fastidieuses vérifications. Et ensuite?

En général dans un programme relativement complexe, l'erreur a lieu dans une fonction, et le mécanisme de gestion de l'erreur a lieu ailleurs dans le code. Comment faire alors pour gérer ces comportements le mieux possible?

Les Exceptions

Il existe en C++(comme en Java et dans d'autres langages) un mécanisme de gestion des erreurs possibles dans un programme. Ce mécanisme est appelé 'Exceptions'.

Par exemple :

Lorsqu'on construit un vecteur, et qu'on demande à l'utilisateur de choisir un indice de ce vecteur, rien ne lui interdit de rentrer un indice négatif, ou un indice trop grand. De même, dans le choix d'un fichier à ouvrir, il se peut que ce fichier n'existe pas. On doit alors gérer le comportement du programme pour qu'il s'adapte à cette possibilité on n'a pas envie que word plante si on ouvre un fichier qui n'existe pas.

Les Exceptions : Un exemple

```
#ifndef __VECTOR_HPP__
#define __VECTOR_HPP__
#include <string>
using namespace std;
template <typename T>
class Vector
{ T * _datas;
int _size;
public:
Vector(int n){ _datas = new
T[n]; _size = n;}
~Vector () { delete [] _datas;}
T& operator [] (int i){
if (i < 0 || i >= _size)
throw (string(" Indice hors limite
: ") + to_string (i));
return _datas[i];}
};
#endif
```

Dans cet exemple, on retrouve la classe **template Vector**. On a juste surchargé l'opérateur [] avec une nouveauté à l'intérieur de celui ci. Il y a maintenant une condition sur l'indice passé à l'opérateur. Lorsque cet indice est négatif, ou que celui ci est trop grand, on lance une exception. Le mot clé qui permet de lancer une exception est **throw**. Celui ci s'utilise conjointement avec une expression d'un type quelconque, utilisateur ou natif.

Les Exceptions : Un exemple

```
#ifndef __VECTOR_HPP__
#define __VECTOR_HPP__
#include <string>
using namespace std;
template <typename T>
class Vector
{ T * _datas;
  int _size;
public:
  Vector(int n){ _datas = new T[n];
    _size = n;}
  ~Vector () { delete [] _datas;}
  T& operator [] (int i){
    if (i < 0 || i >= _size)
      throw (string("Indice hors limite :
") + to_string(i));
    return _datas[i];}
};
#endif
```

Ici, **throw** est suivi d'une **string**, une chaîne de caractère que l'on construit pour décrire l'erreur qui a eu lieu. Même si cette utilisation de **throw** est possible, elle n'est néanmoins pas recommandée. En effet, on verra qu'il vaut mieux utiliser des classes déjà présentes dans la librairie standard.

Les Exceptions : Un exemple

```
#ifndef __VECTOR_HPP__
#define __VECTOR_HPP__
#include <string>
using namespace std;
template <typename T>
class Vector
{ T * _datas;
  int _size;
public:
  Vector(int n){ _datas = new
  T[n]; _size = n;}
  ~Vector () { delete []
  _datas;}
  T& operator [] (int i){
  if (i < 0 || i >= _size)
  throw (string("Indice hors
  limite : ") + to_string (i));
  return _datas[i];}
};
#endif
```

```
#include <iostream>
#include <string>
#include "Vector.hpp"
using namespace std;
int main(){
  vector<double> v(5);
  try{ v[3] = 3;}
  catch (string s){
    cout << s << endl;
  }
  try{
    v[5] = 3;
  }
  catch (string s){
    cout << s << endl;
  }
}
```

Ce n'est pas tout de lancer une exception, il faut aussi pouvoir l'intercepter quelque part. Pour cela, on va utiliser un bloc dit `try{} catch{}`. On place dans le bloc 'try' les instructions qui sont susceptibles de lever une exceptions. Le bloc `catch` lui prend en paramètre une expression d'un type donné. Si l'exception levée est de ce type, alors les instructions de son blocs seront exécutées.

Les Exceptions : Exceptions de type utilisateur

```
#ifndef __A_HH__
#define __A_HH__
#include <string>
using namespace
std;
class Exc
{
public:
Exc(string);
string _message;
};
class A
{
public:
void f();
};
#endif
```

```
#include "A.hh"
Exc::Exc (string s) :
_message(s) {}
void A::f()
{
throw Exc(" KO!");
}
```

```
#include<iostream>
#include "A.hh"
using namespace
std;
int main()
{ A a;
try{
a.f();
}
catch ( Exc e){
cout <<
e._message <<
endl;
}
}
```

On a dit que les exceptions levées pouvaient être de n'importe quel type. Aussi, afin de bien faire la distinction entre les différentes exceptions qui peuvent se déclencher au cours de l'exécution, on peut créer des types différents pour chacune d'elles.

Les Exceptions : Exceptions de type utilisateur

```
#ifndef __A_HH__
#define __A_HH__
#include <string>
using namespace
std;
class Exc
{
public:
Exc(string);
string _message;
};
class A
{
public:
void f();
};
#endif
```

```
#include "A.hh"
Exc::Exc (string s) :
_message(s) {}
void A::f()
{
throw Exc (" KO!");
}
```

```
#include<iostream>
#include "A.hh"
using namespace
std;
int main()
{ A a;
try{
a.f();
}
catch ( Exc e){
cout <<
e._message <<
endl;
}
}
```

Ici, la classe **Exc** va servir à créer un type exception pouvant contenir un message. Ce message sera initialisé lors de la levée de l'exception.

Les Exceptions : Exceptions de type utilisateur

```
#ifndef __A_HH__
#define __A_HH__
#include <string>
using namespace
std;
class Exc
{
public:
Exc(string);
string _message;
};
class A
{
public:
void f();
};
#endif
```

```
#include "A.hh"
Exc::Exc (string s) :
_message(s) {}
void A::f()
{
throw Exc (" KO!");
}
```

```
#include<iostream>
#include "A.hh"
using namespace
std;
int main()
{ A a;
try{
a.f();
}
catch ( Exc e){
cout <<
e._message <<
endl;
}
}
```

Ici, par exemple, lors de la levée (appel de l'instruction **throw** on construit une exception en lui passant une chaîne de caractère en paramètre. Cette chaîne de caractère va pouvoir être affichée dans le bloc **catch** correspondant à cette exception.

Les Exceptions : Plusieurs catch

```
#ifndef __A_HH__
#define __A_HH__
#include <string>
using namespace
std;
class Exc
{
public:
Exc(string);
string _message;
};
class A
{
public:
void f();
void f2();
};
#endif
```

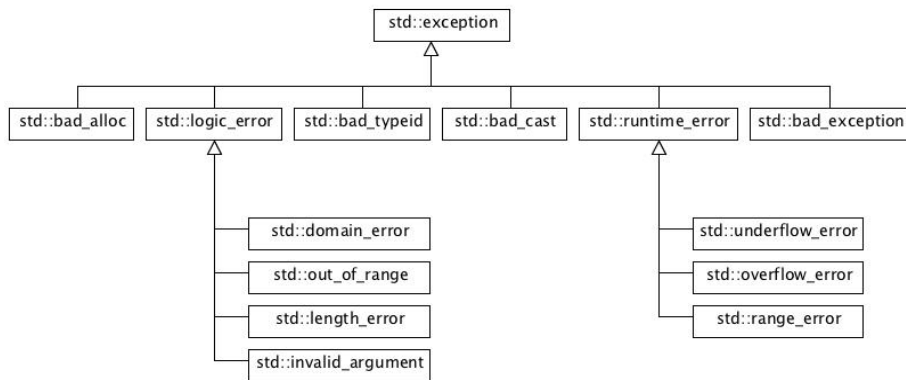
```
#include "A.hh"
Exc::Exc (string s) :
_message(s) {}
void A::f()
{
throw Exc
((" KO!"));
}
void A::f2()
{
throw
string(" Encore
KO");
}
```

```
#include<iostream>
#include "A.hh"
using namespace
std;
int main()
{ A a;
try{
a.f2();
}
catch ( Exc e){
cout <<
e._message <<
endl;
}
catch ( string s){
cout << s <<
endl;
}
}
```

À la suite d'un bloc **try** on peut mettre plusieurs blocs **catch**. Le bloc **catch** correspondant au type de l'exception qui a été levée sera exécuté. Il recevra en paramètre l'exception (passée par valeur depuis le **throw**).

Les Exceptions : Les Exceptions Standard

Il existe des classes d'exceptions déjà disponibles dans la librairie standard. Elles dérivent toutes d'une même classe `<< exception >>`. Cette classe fournit une fonction virtuelle `<< what >>` qui renvoie une chaîne de caractères de type C (`char*`) décrivant l'erreur. Au niveau de l'héritage, elles s'articulent de la façon suivante.



Les Exceptions : Les Exceptions Standard

- On peut soit réutiliser ces classes dans le code, en fonction de l'utilisation de cette classe. Par exemple, ce n'est pas déraisonnable d'utiliser la classe `range_error` dans le cas d'un mauvais indice d'un tableau. Par contre, utiliser `bad_alloc` pour une erreur non liée à de la mémoire serait malvenue.
- On peut également créer soi même des classes qui hériteraient de la classe `exception`, afin qu'elle puisse être gérée comme les autres.

